

BernatLab

Fonaments, contenidors i pràctica

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 1

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el primer mòdul del manual tècnic del BernatLab. Cobreix els fonaments: comprendre la Raspberry Pi, administrar Linux, configurar xarxa i SSH, desplegar contenidors amb Docker, gestionar-los amb Portainer, monitorar-los amb Uptime Kuma, presentar-los amb Homepage, versionar-ho tot amb Git, i dibuixar la full de ruta dels pròxims mesos.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 1. Què és BernatLab
- 2. La Raspberry Pi 4 per dins
- 3. Linux per administrar un servidor
- 4. Xarxa, SSH i Tailscale
- 5. Docker des de zero
- 6. Portainer
- 7. Uptime Kuma
- 8. Homepage
- 9. Git i documentació
- 10. Full de ruta del BernatLab

Capítol 1 — Què és BernatLab

*“Si vols entendre un sistema, construeix-lo tu mateix. Si el vols mantenir, documenta'l.”**

1.1 Què és un homelab

Un **homelab** és un servidor — o un petit conjunt de servidors — que un particular munta a casa seva per aprendre, experimentar i allotjar els seus propis serveis. No és una empresa de hosting. No és un núvol professional. És el teu laboratori personal, la teva escola i, sovint, la teva joguina preferida.

La paraula **home** ja diu molt: el servidor viu a casa teva, normalment connectat al teu router domèstic, darrere d'una IP dinàmica i d'un tallafoç que no controles. La paraula **lab** diu la resta: és un entament d'experimentació, no de producció. Aquí es pot trencar coses. Aquí es poden provar tecnologies noves. Aquí s'equivoca un mateix i s'aprèn d'això.

Un homelab no és cap novetat. Fa dècades que enginyers i administradors de sistemes en tenen: torres amb discs durs, màquines velles reciclades, plaques base en calaixos. El que ha canviat en els últims anys és que avui pots muntar un homelab seriós, capaç de fer feines reals, amb una Raspberry Pi de 50 € i una targeta microSD de 16 €. I el que també ha canviat és el programari: contenidors Docker, serveis autoallotjats, xarxes privades com Tailscale, eines de monitorització com Uptime Kuma, panells com Homepage. Tot plegat, un ecosistema ric que abans era patrimoni de les empreses.

1.2 Per què construir un servidor propi

Hi ha moltes raons per tenir un homelab, i la majoria d'elles tenen poc a veure amb estalviar diners — probablement pagaries menys per un VPS de 3 € al mes. La raó de fons és **aprenentatge i control**.

Quan algú decideix muntar un homelab, el que busca és:

- **Aprendre de veritat.** Llegir sobre Docker no és entendre Docker. Posar-lo en marxa, trencar-lo, refer-lo, llegir els logs a les dues de la matinada, descobrir per què un contenidor no es connecta a una xarxa — això és aprendre.
- **Tenir control sobre les dades.** Quan puges una foto a un núvol, algú altre decideix què se'n fa, durant quant de temps es guarda i qui hi té accés. Quan les dades viuen a la teva Raspberry, la decisió és teva.
- **Experimentar amb projectes reals.** Sensors, automatitzacions, webs personals, bases de dades, petits models d'intel·ligència artificial. Tot plegat, en una sola caixa que cabria en un calaix.
- **Reduir la dependència tecnològica.** Si un dia vols deixar d'utilitzar un servei comercial, tenir la infraestructura a casa et permet fer-ho gradualment.
- **Divertir-se.** Muntar un homelab i mantenir-lo sa és una afició com pot ser-ho la moto, la cuina o la programació. Té un component pràctic, però sobretot un component de plaer personal.

I, francament, hi ha una darrera raó menys noble però igualment vàlida: la **curiositat**. Saber què passa quan un contenidor no arrenca, entendre per què una VPN et permet entrar al teu servidor des de qualsevol lloc del món, descobrir que un missatge de Telegram pot fer arrencar un motor a 245

metres de casa teva. Això, avui, és a l'abast de qualsevol amb una Raspberry Pi i ganes de furgar.

1.3 Què volem aconseguir amb el BernatLab

El BernatLab no és un projecte abstracte. Té objectius molt concrets:

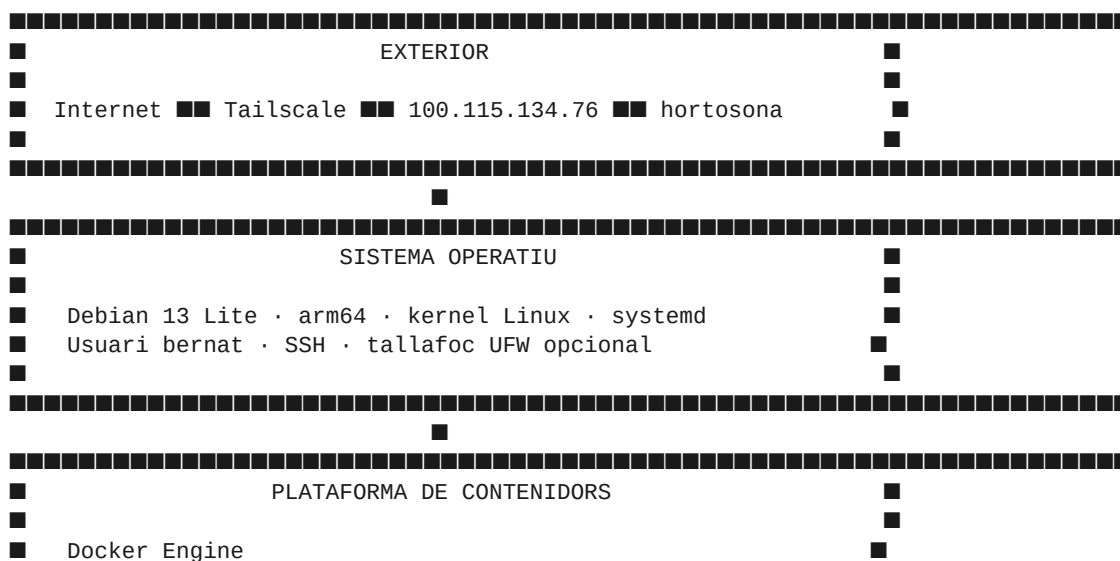
1. **Centralitzar serveis.** En lloc de tenir una Raspberry per als sensors, un servidor vell per a la web, un portàtil per fer proves, tenir una sola màquina que ho aculli tot — o que, com a mínim, en sigui el centre de control.
2. **Donar suport al projecte Hort Osona.** Una aplicació web pública allotjada a GitHub Pages, una API futura allotjada al BernatLab, sensors al terreny que enviaran dades per MQTT, una base de dades InfluxDB, gràfiques amb Grafana.
3. **Servir com a entorn d'aprenentatge continu.** Vull aprendre Docker a fons, entendre xarxes, provar Node-RED, experimentar amb Telegram bots, muntar un assistent IA local amb Ollama. Tot plegat, en un entorn segur i controlat.
4. **Servir com a centre de projectes personals.** Música, fotografia, desenvolupament web, scripts d'automatització, còpies de seguretat. Tot ha de tenir cabuda.
5. **Ser resilient dins les seves limitacions.** Una Raspberry Pi 4 amb 4 GB de RAM no és un servidor de producció. No ha de ser-ho. Ha de ser estable, documentat, fàcil de recuperar i honest amb els seus límits.

El que **no** volem:

- Un servidor que depengui de trucs opacs que ningú entén.
- Un sistema ple de comandes copiades de StackOverflow que ningú sap per què funcionen.
- Un punt únic de fallada que, quan es trenqui, ens deixi sense res.
- Una despesa energètica o econòmica desproporcionada.

1.4 Arquitectura general del BernatLab

A grans trets, el BernatLab té tres capes:



■ ■ ■ ■	Docker Compose (fitxers a /home/bernat/homelab)	■
■ ■ ■ ■	Portainer (gestió web)	■
■ ■ ■ ■	Uptime Kuma (monitorització)	■
■ ■ ■ ■	Homepage (panell d'entrada)	■
■ ■ ■ ■	[pròxims: File Browser, Node-RED, Mosquitto, ...]	■
■ ■ ■ ■		■

Aquesta divisió en tres capes és pedagògica i pràctica alhora. Entenent-les per separat, podem raonar sobre què falla quan alguna cosa falla.

La capa exterior

És el que es veu des de fora. La Raspberry Pi té una IP a la teva xarxa local (normalment `192.168.x.y`), una IP pública dinàmica que el teu operador assigna al router, i una IP Tailscale (`100.115.134.76` en el nostre cas) que és fixa mentre duri la teva subscripció i que ens permet accedir-hi des de qualsevol lloc del món sense tocar el router.

La idea clau és aquesta: **mai no exposem directament la Raspberry a Internet**. Tots els accessos remots passen per Tailscale, que crea una xarxa privada virtual entre els teus dispositius. El router de casa no sap que existeix la Raspberry com a servidor — sap que és un dispositiu més de la xarxa local.

La capa de sistema operatiu

Debian 13 Lite és la base. La versió Lite no porta entorn gràfic, ni servidor d'impressió, ni mil programes innecessaris. Porta just el que un servidor necessita: el kernel, les utilitats bàsiques, el gestor de paquets `apt`, el dimoni `systemd` que arrenca els serveis, i poc més. Això és bo perquè redueix la superfície d'atac i el consum de RAM.

Aquí és on vivim nosaltres com a administradors: editem fitxers, instal·lem programes, configurem serveis de sistema, gestionem usuaris, mirem logs.

La capa de contenidors

Per sobre del sistema operatiu, Docker. Docker ens permet instal·lar serveis complexos — com Portainer, Uptime Kuma, Homepage, Grafana — sense contaminar el sistema base. Cada servei viu en un **contenedor**: un entorn aïllat que conté el seu propi sistema de fitxers, la seva pròpia xarxa i el seu propi cicle de vida. Si un contenidor es trenca, l'esborres, el tornes a crear, i tot segueix igual. Si vols actualitzar un servei, fas una ordre i Docker es descarrega la nova versió, atura l'antic i engega el nou.

La capa de contenidors es gestiona amb **Docker Compose**, que és un fitxer `yaml` on descrius tots els teus serveis, les seves configuracions, els volums, les xarxes i els ports. En lloc de recordar vint ordres llargues, tens un sol fitxer que documenta tot el sistema. A `/home/bernat/homelab/docker-compose.yaml` hi haurà la definició de tot el BernatLab.

1.5 Serveis que ja tenim en marxa

A dia d'avui, el BernatLab té tres serveis principals desplegats amb Docker Compose:

Portainer (<https://100.115.134.76:9443>)

Portainer és una interfície web que ens permet veure i gestionar tots els contenidors Docker de la Raspberry sense haver de tocar la línia d'ordres. És com un panell de control per al Docker. Porta un any, dos, sent l'eina estàndard per a homelabs i petites empreses.

Hi accedim per HTTPS, tot i que el certificat és autosignat (ens saltarà l'avís del navegador la primera vegada, cosa perfectament normal en un entorn personal). Un cop dins, podem veure contenidors actius, aturar-los, reiniciar-los, veure els logs en directe, explorar volums, veure imatges, gestionar xarxes. També permet crear piles (stacks) de Docker Compose directament des del navegador, cosa que pot ser útil però que, per a aquest manual, farem sempre des de la línia d'ordres per entendre què passa.

Uptime Kuma (<http://100.115.134.76:3001>)

Uptime Kuma és una eina de monitorització. La seva feina és senzilla: comprovar periòdicament que els nostres serveis estan vius i avisar-nos quan no ho estan. Pot fer pings a màquines, peticions HTTP a webs, comprovacions de ports TCP, comprovacions de certificats SSL, i un llarg etcètera.

En el BernatLab, Uptime Kuma vigila, com a mínim:

- La pròpia Raspberry Pi (ping).
- Portainer (HTTP).
- El mateix Uptime Kuma (meta-monitorització).
- La web pública Hort Osona (HTTP a bernadmora.github.io).

Quan algun d'aquests serveis falla, Uptime Kuma pot enviar una alerta per correu, Telegram, Discord, Slack, webhook, o qualsevol combinació imaginable. En aquest manual veurem com configurar-lo per avisar-nos per Telegram.

Homepage (<http://100.115.134.76:3000>)

Homepage és el **panell d'entrada** al BernatLab. Una pàgina web molt neta, feta per un projecte de codi obert, que ens mostra una graella de targetes: cadascuna és un enllaç directe als nostres serveis. L'objectiu és que quan obrim el navegador i posem `100.115.134.76:3000`, veiem d'un cop d'ull tot el que tenim, si està funcionant, i hi puguem entrar amb un sol clic.

Homepage és personalitzable: podem posar un fons, podem agrupar serveis per categories (Monitorització, Gestió, Dades, Experimentals, etc.), podem afegir widgets que mostren estadístiques en directe (ús de CPU, RAM, temperatura, etc.). És la porta d'entrada i la targeta de presentació del BernatLab.

1.6 Filosofia del projecte

El BernatLab es regeix per cinc principis. No són normes escrites en marbre, però ajuden a prendre decisions quan hi ha dubtes.

Primer principi: entendre abans de copiar

Quan trobem una ordre a Internet, abans d'executar-la a cegues, l'hem d'entendre. Què fa? Per què funciona? Què canviaria si la meua màquina fos diferent? Si no podem respondre aquestes preguntes, millor preguntar, provar en un entorn segur o esperar. Un homelab és, sobretot, una escola.

Segon principi: documentar sempre

Si hem après alguna cosa, l'escrivim. Si hem trobat un error, l'escrivim. Si hem canviat una configuració, l'escrivim. El futur jo del Bernat agrairà el present jo del Bernat cada vegada que obri un fitxer i hi trobi el context que necessita.

Tercer principi: senzillesa per defecte

Si hi ha dues maneres de fer una cosa, escollim la més senzilla. Un sol contenidor és millor que dos. Un fitxer de configuració és millor que tres. Una eina bona és millor que tres eines mitjanes. La complexitat és la principal font d'errors en un homelab.

Quart principi: còpies de seguretat com a rutina

Un homelab sense còpies de seguretat és una bomba de rellotgeria. Les targetes microSD fallen. Els discs durs fallen. Les actualitzacions fallen. Si no podem restaurar el sistema en una hora, no estem preparats. Per això, en el Capítol 9, dedicarem temps a còpies de seguretat de la carpeta `/home/bernat/home\lab` i de les bases de dades.

Cinquè principi: mesurar abans d'optimitzar

No afegim RAM perquè sí. No comprem un disc SSD perquè ens sembla. No movem serveis a una màquina més potent per intuïció. Primer mesurem — Uptime Kuma, `htop`, `docker stats`, logs — i després decidim.

1.7 Com s'administra un homelab: el dia a dia

Un homelab no és un projecte que es fa un cop i s'oblida. És un sistema viu que canvia, creix i, de tant en tant, es trenca. La vida d'un administrador de homelab es compon de tasques quotidianes que, amb el temps, esdevenen rutines:

- **Matí:** comprovar Uptime Kuma, mirar si hi ha alertes.
- **Setmanal:** actualitzar contenidors, llegir notes de versió, fer còpia de seguretat.
- **Quan toca:** afegir un servei nou, configurar un sensor nou, muntar una base de dades.
- **Quan es trenca:** mirar logs, buscar l'error, arreglar-lo, documentar-lo.

Aquest manual està pensat per donar-te les eines per fer tot això amb confiança. No per tenir un servidor bonic, sinó per tenir un servidor **entès**.

1.8 Resum

En aquest primer capítol hem après què és un homelab, per què val la pena tenir-ne un, quins objectius concrets té el BernatLab, com s'estructura en tres capes (xarxa, sistema operatiu, contenidors) i quins serveis hi ha desplegats. També hem establert cinc principis que ens guiaran: entendre, documentar, simplificar, copiar i mesurar. En el proper capítol baixarem al metall: la Raspberry Pi 4, peça a peça.

1.9 Exercicis pràctics

1. **Obre el panell d'Homepage** (<http://100.115.134.76:3000>) i mira què hi ha. Anota tres serveis que vulguis afegir.
2. **Obre Portainer** (<https://100.115.134.76:9443>) i compta quants contenidors estan **running**. Apunta'ls tots en un paper.
3. **Obre Uptime Kuma** (<http://100.115.134.76:3001>) i mira l'estat dels monitors. N'hi ha cap de caigut? Si sí, investiga per què.
4. **Connecta't per SSH** a la Raspberry i executa `hostname`, `uptime` i `whoami`. Apunta el resultat. Això ja és administració de veritat.

Comandes útils d'aquest capítol:

```
ssh bernat@100.115.134.76
hostname
uptime
whoami
```

Paraules clau per recordar: **homelab**, **autosuficiència**, **contenidors**, **documentació**, **mesurar**, **simplificar**.

Capítol 2 — La Raspberry Pi 4 per dins

"El primer pas per administrar bé una màquina és entendre-la. La Raspberry Pi 4 no és un PC: és un ordinador amb coses per aprendre."

2.1 Què és exactament una Raspberry Pi 4

La Raspberry Pi 4 Model B és un **ordinador de placa única** (SBC, single-board computer) fabricat per la Raspberry Pi Foundation, una organització britànica sense ànim de lucre que va néixer el 2012 amb l'objectiu de portar la informàtica a les escoles. La idea era construir un ordinador prou barat, prou petit i prou obert perquè qualsevol nen o nena del món pogués aprendre programació. Aquella idea va acabar convertint-se en una plataforma de referència per a projectes de tota mena: des de media centers fins a homelabs, passant per robots, càmeres de seguretat, sistemes industrials i, naturalment, servidors personals com el BernatLab.

La Raspberry Pi 4 va aparèixer el 2019 com la quarta generació important de la família. La **B** del nom indica que és la versió completa, amb tots els ports i capacitats — la versió que ens interessa per a un servidor. A diferència de la Raspberry Pi 5 (més nova, més cara, més potent), la 4 té un equilibri excel·lent entre preu, consum i potència, i la comunitat la coneix a fons.

El model que tenim al BernatLab és la **Raspberry Pi 4 Model B amb 4 GB de RAM**. Hi ha versions amb 1 GB, 2 GB i 8 GB; 4 GB és el punt dolç per a un servidor amb contenidors: prou memòria per a Portainer, Uptime Kuma, Homepage, i encara queda espai per afegir Grafana, InfluxDB o una base de dades petita.

2.2 CPU: el cor ARM

La CPU és el **Broadcom BCM2711**, un processador **ARM Cortex-A72** de quatre nuclis (quad-core) a 1,8 GHz. Això és molt important per entendre la Raspberry Pi, perquè ARM no és x86.

La majoria d'ordinadors que has fet servir — PCs de sobretaula, portàtils, servidors professionals — duen processadors **x86** (Intel o AMD), una arquitectura de 64 bits amb instruccions complexes (CISC). Les Raspberry Pi duen processadors **ARM**, una arquitectura de 32 o 64 bits amb instruccions reduïdes (RISC), pensada originalment per a mòbils i sistemes encastats. La diferència pràctica, avui, és menor del que sembla: les dues arquitectures corren Linux, les dues corren Docker, les dues corren navegadors moderns. Però hi ha tres coses que cal saber:

1. **Les imatges Docker arm64:** quan descarreguem una imatge de Docker Hub, hem de mirar si n'hi ha versió `linux/arm64` (la nostra) o només `linux/amd64` (x86). La majoria de projectes moderns ja publiquen ambdues. Si una imatge només té `amd64`, Docker pot intentar executar-la amb **emulació** (`binfmt_misc`) i funcionar, però amb una pèrdua de rendiment notable.
2. **El sistema operatiu és arm64:** per això tenim **Debian 13 Trixie Lite per a arm64**. Si algú ens passa una ISO per a x86, no funcionarà. Si volem instal·lar programes, hem d'assegurar-nos que el paquet `apt` és per a arm64 (cosa que el propi `apt` ja gestiona quan detecta l'arquitectura).
3. **Rendiment:** a 1,8 GHz amb quatre nuclis, la CPU és modesta comparada amb un PC modern, però més que suficient per a un servidor de serveis petits. La limitació real no és la CPU; és la RAM i, sobretot, l'emmagatzematge.

Els quatre nuclis volen dir que el sistema pot fer quatre coses alhora de veritat, no de manera simulada. Això és útil per a Docker: un contenidor pot estar fent una consulta a base de dades mentre un altre serveix pàgines web i un tercer monitoritza. En un sistema amb un sol nucli, tot aniria molt més a poc a poc.

2.3 Memòria RAM

Tenim 4 GB de RAM LPDDR4. La LPDDR4 és una memòria de baix consum dissenyada originalment per a mòbils, que comparteix el mateix xip que la CPU (package-on-package, PoP). Això té avantatges (menys latència, menys espai) i inconvenients (no es pot ampliar, no es pot canviar).

Quan el sistema arrenca, Debian Lite consumeix al voltant de 80-120 MB de RAM. La resta queda lliure per a aplicacions i contenidors Docker. Una distribució amb entorn gràfic consumiria 400-600 MB només per al sistema base, la qual cosa deixaria molt poc espai per als serveis.

Amb 4 GB podem allotjar còmodament 5-8 contenidors petits simultàniament. Si volem allotjar InfluxDB amb una càrrega important de dades, Grafana, una base de dades PostgreSQL, i mitja dotzena més de serveis, començarem a notar la pressió. Per això, en el Capítol 10, parlarem de la possibilitat d'afegir swap a un disc SSD extern — una mena de "RAM virtual" que ens pot treure d'un embús puntual, tot i que és molt més lenta que la RAM real.

Per veure la memòria disponible, podem executar:

```
free -h
```

La sortida mostrarà quatre línies importants: **Mem** (memòria física total), **Swap** (disc d'intercanvi), amb columnes **total**, **used**, **free** i **available**. La que realment importa és **available**, que és la que el nucli considera utilitzable tenint en compte caches i reserves.

2.4 Emmagatzematge: la targeta microSD

Aquí hi ha la gran particularitat de la Raspberry Pi — i el seu principal punt feble com a servidor. L'emmagatzematge principal és una **targeta microSD**, no un disc dur ni un SSD.

La Raspberry Pi 4 no porta emmagatzematge intern. Per arrencar, necessita una targeta microSD amb un sistema operatiu gravat. A la pràctica, una microSD és una memòria flash NAND — la mateixa tecnologia que duen els pendrives USB i els discs SSD barats — però amb un controlador molt més senzill i, per tant, amb una durabilitat molt menor.

Això vol dir que les targetes microSD:

- **Es degraden amb les escriptures.** Cada vegada que el sistema escriu un fitxer, la memòria NAND pateix un petit desgast. Les targetes modernes duren milers de cicles d'escriptura, però un servidor escriu molt: logs, bases de dades, configuracions, actualitzacions. Una targeta barata pot morir en mesos.
- **Són lentes.** Comparades amb un SSD, les microSD tenen latències altes i velocitats seqüencials baixes. Per a un servidor amb molta lectura (servir pàgines web, executar consultes), això es nota.
- **Són petites.** 16, 32, 64 GB, rarament més. Per a un sistema operatiu amb contenidors, això s'omple ràpid.

La solució estàndard al món Raspberry és **arrencar des d'un disc SSD USB**. La Raspberry Pi 4 ho permet des del firmware de 2020: amb una actualització de la EEPROM i un disc SSD connectat a un port USB 3.0, podem fer que la màquina arrenqui des del SSD com si fos una microSD. Aquesta és una de les millors que tenim a la full de ruta (Capítol 10) — encara no implementada, però prevista.

Mentre no fem el pas a SSD, podem allargar la vida de la microSD amb dues mesures bàsiques:

1. **Usar una targeta de qualitat**. Samsung EVO, SanDisk Extreme, Kingston Canvas. Targeta "no name" de 5 € és llençar diners.
2. **Reduir les escriptures**. Moure els logs a RAM amb `log2ram`, moure els volums Docker a un SSD extern, desactivar la swap agressiva, evitar `apt` innecessaris.

2.5 Ports i connectivitat

La Raspberry Pi 4 Model B té una quantitat interessant de ports, alguns dels quals val la pena conèixer bé:

USB

Dos ports **USB 3.0** (blaus) i dos ports **USB 2.0** (negres). Els USB 3.0 són útils per connectar un SSD extern (fins a 5 Gbps teòrics, ~400 MB/s reals) o dispositius d'emmagatzematge ràpids. Els USB 2.0 (480 Mbps) són per a perifèrics lents: teclats, ratolins, dongles Wi-Fi externs, o un segon disc si la velocitat no és crítica.

Ethernet

Un port **Gigabit Ethernet** (1 Gbps, 125 MB/s) connectat directament al xip, no pas a un hub USB intern (a diferència de les Raspberry anteriors). Això vol dir que la xarxa per cable pot aprofitar-se bé: podem saturar-la amb un SSD de xarxa o amb tràfic intens. Per a un servidor personal, 1 Gbps és més que suficient: la majoria d'Internet domèstic no passa de 100-300 Mbps de baixada.

Wi-Fi i Bluetooth

La Raspberry Pi 4 porta un xip **Cypress CYW43455** que ofereix Wi-Fi 802.11ac (dual-band 2,4 i 5 GHz) i Bluetooth 5.0. Per a un servidor, la Wi-Fi és útil com a backup si el cable falla, però **un servidor sempre ha d'anar per cable**. La Wi-Fi és menys estable, té més latència, i en un homelab volem predibilitat.

GPIO

40 pins **GPIO** (general-purpose input/output) són el cor del "maker" de la Raspberry. A través d'aquests pins podem connectar sensors, LEDs, motors, relés, i comunicar-nos amb ells des de programes. Són la porta d'entrada al món de l'electrònica.

En el context del BernatLab, els GPIO els usarem eventualment per connectar sensors ambientals o per comunicar-nos amb un mòdul LoRa SX1262. Però aquesta és feina del Capítol 10, no d'ara. De moment, només cal saber que hi són.

HDMI

Dos ports **micro-HMI** (HDMI 0 i HDMI 1) que permeten connectar dos monitors 4K a 30 Hz o un a 60 Hz. Com que tenim la versió Lite de Debian sense entorn gràfic, aquests ports no els farem servir. Sí que els podem fer servir puntualment per connectar un monitor en cas d'emergència — per exemple, si la màquina no arrenca i volem veure què passa.

USB-C (alimentació)

L'**alimentació** entra per un connector USB-C. La Raspberry Pi 4 accepta 5 V a 3 A, és a dir, 15 W de potència màxima. En un servidor, és recomanable usar un carregador oficial o un de qualitat, capaç de subministrar els 3 A de forma estable. Subministrar menys pot provocar reinicions sota càrrega, cosa que en un servidor és inadmissible.

2.6 Temperatura i refrigeració

La CPU genera calor. Sense refrigeració, sota càrrega, pot arribar a 80-85 °C i començar a fer **thermal throttling**: reduir la velocitat dels nuclis per no cremar-se. Això passa amb freqüència en servidors 24/7.

Les opcions de refrigeració són:

- **Disipador passiu**: un petit bloc d'alumini enganxat al xip. Gratuït o gairebé, funciona bé si la càrrega és moderada.
- **Ventilador actiu**: un petit ventilador de 30 mm alimentat pels pins GPIO 5V. Gairebé sempre inaudible, refreda molt bé, però afegeix un component mecànic que es pot trencar.
- **Carcassa amb dissipació**: una carcassa d'alumini que actua com a dissipador. Elegant, però més cara.

Al BernatLab, tenim [pendent d'anotar quina solució hem posat]. Independentment del que hi hagi, podem monitoritzar la temperatura amb:

```
vcgencmd measure_temp
```

o, si `vcgencmd` no està disponible:

```
cat /sys/thermal/thermal_zone0/temp
```

que retorna la temperatura en mil·lis de grau Celsius (dividir entre 1000).

Si la temperatura supera els 70 °C de forma regular, és hora de millorar la refrigeració. En un servidor, la temperatura òptima és 40-60 °C en repòs, 55-70 °C sota càrrega. Més de 80 °C és senyal d'alarma.

2.7 Procés d'arrencada

Quan connectem l'alimentació a la Raspberry Pi 4, passa el següent:

1. **El xip de vídeo (VideoCore VII) agafa el control**. Llegeix la primera etapa del bootloader des d'una petita ROM interna del xip. Aquesta primera etapa és immutable i s'encarrega d'inicialitzar el mínim necessari.
2. **El bootloader carrega la segona etapa** des de la targeta microSD (o SSD USB). Aquesta segona etapa és a la partició `boot` (FAT32) i s'anomena `bootcode.bin` o, en versions modernes, és el

fitxer `pieeprom.bin`.

3. **La EEPROM entra en acció.** En les Raspberry Pi 4 modernes, el bootloader es troba en una EEPROM reprogramable. Això ens permet actualitzar el firmware (per exemple, per habilitar l'arrencada USB) sense canviar la targeta.

4. **El kernel Linux arrenca.** Un cop el bootloader ha acabat la seva feina, carrega el fitxer `kernel8.img` (per a arm64) i li passa el control. El kernel munta el sistema de fitxers arrel (/), carrega els mòduls, configura el maquinari.

5. **systemd arrenca els serveis.** Un cop el kernel ha muntat tot el que cal, executa `/sbin/init`, que en el nostre cas és un enllaç a `systemd`. `systemd` és el **gestor de serveis i d'arrencada** que s'encarrega d'activar tots els serveis del sistema: xarxa, SSH, Docker, Portainer, etc.

6. **El sistema està llest.** Tots els serveis configurats per arrencar automàticament estan funcionant. La Raspberry està esperant connexions SSH, contenidors en marxa, etc.

El procés complet dura entre 15 i 60 segons, depenent de la targeta i de quants serveis tinguis. Ho podem comprobar amb:

```
systemd-analyze
```

que ens dirà quant ha trigat el kernel i quant ha trigat l'espai d'usuari. Si triga massa, podem veure on es perd el temps amb:

```
systemd-analyze blame
```

que ens dóna una llista dels serveis ordenats pel temps que han trigat a arrencar. Si Docker triga 20 segons i Portainer 8, potser podem optimitzar l'ordre d'arrencada.

2.8 El kernel Linux

El **kernel** és el nucli del sistema operatiu. És el programa que s'executa amb més privilegis (mode kernel) i que té accés directe al maquinari: CPU, memòria, discos, xarxa, GPIO. Tot el que fem servir — la línia d'ordres, els contenidors, els serveis — passa per sobre del kernel, que actua com a intermediari entre el programari i el metall.

Al BernatLab tenim un **kernel Linux** compilat per a ARM, versió [caldrà comprovar amb `uname -r`]. Aquest kernel inclou tots els controladors necessaris per al maquinari de la Raspberry Pi 4: el controlador de la CPU, de la xarxa, de l'USB, del GPIO, de la GPU, etc.

El kernel es pot actualitzar amb `apt`:

```
sudo apt update
sudo apt full-upgrade
```

Però compte: actualitzar el kernel pot trencar coses. Algunes funcionalitats — com el controlador de GPIO, l'accés a la càmera, o la sortida HDMI — poden canviar de comportament entre versions. Per això, en un servidor, sempre és bona idea llegir les notes de versió abans d'actualitzar el kernel, i tenir una manera de tornar enrere si alguna cosa falla.

Per veure quin kernel tenim:

```
uname -a
```

que ens retornarà una línia amb el nom del kernel, la versió, l'arquitectura, la data de compilació i el hostname.

2.9 systemd: el mestre de cerimònies

systemd és el **gestor de sistemes i serveis** que Debian i la majoria de distribucions modernes fan servir. La seva feina és:

- Arrencar els serveis en l'ordre correcte durant l'inici del sistema.
- Aturar-los ordenadament durant l'apagada.
- Supervisar els serveis i reiniciar-los automàticament si fallen.
- Gestionar punts de muntatge, dispositius, temporitzadors, sockets.

systemd organitza els serveis en **unitats** (units). Cada unitat és un fitxer de text amb una extensió que indica el tipus: `.service` (un servei normal), `.timer` (un temporitzador), `.mount` (un punt de muntatge), `.socket` (un socket de xarxa), `.target` (un objectiu que agrupa altres unitats), etc.

Per exemple, el servei de Docker és `docker.service`, definit a `/lib/systemd/system/docker.service`. Per veure'n l'estat:

```
systemctl status docker
```

Per activar-lo a l'arrencada:

```
sudo systemctl enable docker
```

Per iniciar-lo ara:

```
sudo systemctl start docker
```

Per aturar-lo:

```
sudo systemctl stop docker
```

Per reiniciar-lo:

```
sudo systemctl restart docker
```

Per veure tots els serveis actius:

```
systemctl list-units --type=service --state=running
```

Aquestes ordres les farem servir cada setmana. Són la base de l'administració d'un servidor Linux modern.

2.10 Per què Debian Lite

Triar un sistema operatiu per a un homelab és una decisió important. Tenim opcions com Raspberry Pi OS (l'oficial), Ubuntu Server, DietPi, Arch Linux ARM, Alpine. Per què Debian Lite?

Tres raons pràctiques:

1. **Estabilitat.** Debian és coneguda per ser extremadament estable. La versió actual (Trixie, 13) porta programes provats i madurs, no les últimes versions. Per a un servidor, això és bo: volem que el sistema no canviï sota els nostres peus.
2. **Repositoris.** Debian té un dels repositoris de paquets més grans del món. Qualsevol cosa que necessitem — eines de xarxa, editors, llenguatges de programació, biblioteques — hi és. La majoria de documentació que trobem a Internet assumeix Debian/Ubuntu.

3. Lleugeresa. La versió **Lite** (o **netinst**) no porta entorn gràfic ni programari innecessari. Només el sistema base. Això ens dóna control absolut sobre què hi ha instal·lat i redueix la superfície d'atac.

Raspberry Pi OS, la distribució oficial, és bona i està optimitzada per al maquinari, però porta moltes coses innecessàries per a un servidor (escriptori, Chromium, LibreOffice, Mathematica). DietPi és excel·lent per a sistemes molt limitats, però el seu valor afegit (instal·lador de programes automatitzat) no el necessitem en un entorn on ja tenim Docker. Ubuntu Server seria una bona opció, però la seva política d'actualitzacions és més agressiva i les seves versions LTS canvien coses cada pocs anys.

Debian Lite és, simplement, la millor relació entre simplicitat, estabilitat i control.

2.11 Esquema de la Raspberry Pi 4

Esquema Mermaid (text):

```
graph TB
    subgraph SoC["SoC Broadcom BCM2711"]
        CPU["CPU ARM Cortex-A72<br/>4 nuclis @ 1.8 GHz"]
        GPU["GPU VideoCore VI"]
        RAM["RAM LPDDR4<br/>4 GB"]
    end

    subgraph Xarxa["Connectivitat"]
        ETH["Ethernet<br/>1 Gbps"]
        WIFI["Wi-Fi ac + BT 5.0"]
    end

    subgraph Ports["Ports externs"]
        USB3["2x USB 3.0"]
        USB2["2x USB 2.0"]
        GPIO["40 pins GPIO"]
        HDMI["2x micro-HDMI"]
        USB_C["USB-C<br/>Alimentació 5V/3A"]
    end

    subgraph Emm["Emmagatzematge"]
        SD["microSD<br/>(sistema operatiu)"]
        SSD["SSD USB<br/>(pròximament)"]
    end

    CPU <--> RAM
    CPU <--> GPU
    CPU --> ETH
    CPU --> WIFI
    CPU --> USB3
    CPU --> USB2
    CPU --> GPIO
    GPU --> HDMI
    USB_C --> CPU
    SD --> CPU
    SSD -. -> |pròximament| USB3
```

2.12 Errors habituals

Error 1: comprar una microSD barata. Síntoma: la targeta falla al cap de 2-3 mesos, el sistema es corromp. Solució: targeta de marca (Samsung EVO Select, SanDisk Extreme), classe A2 o superior.

Error 2: subministrar poca potència. Síntoma: la Raspberry es reinicia aleatòriament, sobretot quan hi ha un USB connectat. Solució: carregador oficial o de qualitat, 5V/3A mínim.

Error 3: ignorar la temperatura. Síntoma: rendiment baix, "throttling" als logs (`dmesg | grep -i thermal`). Solució: afegir dissipador, ventilador, o carcassa dissipativa.

Error 4: actualitzar el kernel sense previ avís. Síntoma: el GPIO, la càmera, o algun altre maquinari deixa de funcionar. Solució: llegir notes de versió, fer còpia de seguretat, tenir un pla per tornar enrere.

Error 5: confondre ARM amb x86. Síntoma: descarregar una ISO o una imatge Docker que no funciona. Solució: verificar sempre l'arquitectura amb `uname -m` (que retorna `aarch64` per a `arm64`) i usar imatges adequades.

2.13 Resum

Hem recorregut la Raspberry Pi 4 peça a peça: la CPU ARM Cortex-A72, els 4 GB de RAM, la microSD i les seves limitacions, els ports USB, Ethernet, GPIO i HDMI, l'alimentació USB-C, la temperatura, el procés d'arrencada, el kernel Linux i systemd. Hem entès per què hem triat Debian Lite i quin paper juga cada peça en el BernatLab. En el proper capítol baixarem al sistema operatiu: aprendrem a administrar Linux de veritat.

2.14 Exercicis pràctics

1. Executa `uname -a` i anota la versió exacta del kernel.
2. Executa `free -h` i calcula quanta RAM queda lliure amb tots els serveis aturats.
3. Executa `vcgencmd measure_temp` o `cat /sys/thermal/thermal_zone0/temp` per veure la temperatura actual.
4. Executa `systemd-analyze` per veure quant ha trigat a arrencar el sistema.
5. Executa `systemd-analyze blame` i identifica els tres serveis que més triguen a arrencar.
6. Mira els logs del kernel amb `dmesg | less` i busca missatges sobre el maquinari (Ethernet, USB, etc.).

Comandes útils del capítol:

```
uname -a
free -h
vcgencmd measure_temp
cat /sys/thermal/thermal_zone0/temp
systemd-analyze
systemd-analyze blame
systemctl status docker
systemctl list-units --type=service --state=running
```

Paraules clau: **ARM Cortex-A72, BCM2711, microSD, EEPROM, systemd, kernel, thermal throttling, Debian Lite, arm64.**

Capítol 3 — Linux per administrar un servidor

"La consola no és una eina del passat. És la manera més precisa, més lleugera i més fiable de parlar amb un servidor."

3.1 Per què la consola

Quan algú entra per primera vegada a un servidor Linux, sovint se sent perdut. La pantalla és negra, no hi ha icones, no hi ha menú. Però aquesta pantalla negra és, paradoxalment, una de les eines més potents que tenim. La consola de Linux ens permet:

- Fer coses que una interfície gràfica no pot fer.
- Automatitzar tasques amb un sol script.
- Treballar remotament, per una connexió de xarxa molt petita, sense dependre d'entorns gràfics.
- Entendre què està passant realment, sense la intermediació d'una interfície que amaga la complexitat.

Aquest capítol no és un manual exhaustiu de Linux. És un **recorregut pràctic pels conceptes i les ordres que farem servir cada dia al BernatLab**. Algunes són molt bàsiques, d'altres una mica més avançades. Totes, però, són útils.

3.2 El sistema de fitxers

Tot a Linux és un fitxer. Aquesta és potser la idea més important del sistema. Els directoris són fitxers. Els dispositius de maquinari són fitxers. Els processos són fitxers. Les connexions de xarxa són fitxers. Quan entens això, comences a entendre Linux.

L'estructura del sistema de fitxers de Linux segueix l'estàndard **FHS (Filesystem Hierarchy Standard)**. A Debian, l'estructura arrel és:

```
/
■■■ bin/      → binaris essencials (ls, cat, cp...)
■■■/sbin/    → binaris d'administració (mount, fdisk...)
■■■ etc/      → fitxers de configuració
■■■ home/     → directoris personals dels usuaris
■   ■■■ bernat/ → el nostre directori
■       ■■■ homelab/ → la carpeta de treball del BernatLab
■■■ var/      → dades variables (logs, bases de dades, cues)
■   ■■■ log/   → logs del sistema
■■■ tmp/      → fitxers temporals
■■■ usr/      → programes d'usuari (la majoria del sistema)
■■■ opt/      → programes opcionals / comercials
■■■ lib/      → biblioteques compartides
■■■ dev/      → dispositius
■■■ proc/     → informació del kernel i processos
■■■ sys/      → interfície amb el kernel
■■■ mnt/ i media/ → punts de muntatge temporals
```

Les normes no escrites que val la pena recordar:

- **Tot el que és configurable és a `/etc`**. Si vols canviar el comportament d'un servei, és gairebé segur que el fitxer és aquí.
- **Els logs són a `/var/log`**. Si alguna cosa falla, és aquí on has de mirar.
- **Les dades temporals són a `/tmp`**. Es poden esborrar en qualsevol moment — no hi posis res important.
- **La teva àrea personal és a `/home/bernat`**. Aquí és on treballaràs, on guardaràs els teus projectes, on crearàs la carpeta `homeLab`.

Navegació bàsica

```
pwd          # mostra on sóc
ls           # llista el contingut
ls -l       # llista amb detalls
ls -la      # llista amb detalls i amagats
cd /home/bernat # canvia de directori
cd ..       # puja un nivell
cd ~        # va a /home/bernat
cd -        # torna a l'últim directori
```

Operacions amb fitxers

```
cp origen desti      # copia
mv origen desti      # mou o reanomena
rm fitxer            # esborra
rm -r directori      # esborra recursivament
mkdir directori      # crea directori
rmdir directori      # esborra directori buit
touch fitxer         # crea fitxer buit o actualitza data
cat fitxer           # mostra contingut
less fitxer          # mostra contingut paginat
head -n 20 fitxer    # primeres 20 línies
tail -n 20 fitxer    # últimes 20 línies
tail -f fitxer       # últimes línies, en directe
```

Comandes molt útils al BernatLab

```
du -sh /home/bernat/homelab # espai ocupat
df -h                       # espai en discos
find / -name "docker-compose.yml" # buscar un fitxer
grep -ri "error" /var/log/    # buscar text en fitxers
```

3.3 Usuaris i permisos

Linux és un sistema **multi-usuari**. Cada persona (o cada servei) té un compte propi, amb un nom, un directori personal i uns permisos. Al BernatLab tenim, com a mínim, dos usuaris importants:

- **root**: el superusuari, que pot fer tot. Compte — el seu directori personal és `/root`, no pas `/home/root`.
- **bernat**: l'usuari amb què treballem normalment. Té el seu directori a `/home/bernat`.

Per canviar temporalment a root:

```
su -
```

o per executar una sola ordre amb privilegis de root:

```
sudo ordre
```

`sudo` és el mecanisme estàndard. Està configurat al fitxer `/etc/sudoers` (millor no editar-lo a mà; usa `visudo` per seguretat). L'usuari `bernat` està al grup `sudo`, cosa que li permet executar ordres com a root sense contrasenya o amb la seva pròpia contrasenya.

Permisos

Cada fitxer té tres tipus de permisos per a tres categories d'usuaris:

```
-rwxr-xr-- 1 bernat bernat 1234 oct  3 12:00 script.sh
```

Desglossant:

- `-` → és un fitxer regular (un directori seria `d`)
- `rwx` → el propietari pot llegir, escriure, executar
- `r-x` → el grup pot llegir i executar (no escriure)
- `r--` → la resta només pot llegir

Per canviar permisos:

```
chmod 755 script.sh      # rwxr-xr-x
chmod u+x script.sh      # afegeix execució al propietari
chmod go-w directori     # treu escriptura a grup i altres
```

Per canviar propietari:

```
chown bernat:bernat fitxer
chown -R bernat:bernat directori
```

Els números de `chmod` es calculen així: `r=4`, `w=2`, `x=1`. Suma'ls per a cada categoria. `755` = propietari 7 (`rw`x), grup 5 (`r-x`), altres 5 (`r-x`).

3.4 El sistema de paquets apt

apt és el gestor de paquets de Debian. La seva feina és descarregar, instal·lar, actualitzar i eliminar programes des dels **repositoris** oficials de Debian — enormes magatzems de programari mantinguts per la comunitat.

Configuració bàsica

Els repositoris estan definits a `/etc/apt/sources.list` i als fitxers de `/etc/apt/sources.list.d/`. Si obrim el primer amb `cat`, veurem una cosa semblant a:

```
deb http://deb.debian.org/debian trixie main contrib non-free non-free-firmware
deb http://deb.debian.org/debian trixie-updates main contrib non-free non-free-firmware
deb http://security.debian.org/debian-security trixie-security main contrib non-free non-free-firmware
```

Això vol dir: "Baixa paquets per a la versió Trixie, des del servidor principal de Debian, de les seccions main, contrib, non-free i non-free-firmware".

Ús diari

```

sudo apt update           # actualitza la llista de paquets disponibles
sudo apt upgrade          # actualitza tots els paquets
sudo apt full-upgrade     # igual però permet canvis majors
sudo apt install paquet   # instal·la un paquet
sudo apt remove paquet    # elimina un paquet
sudo apt purge paquet     # elimina un paquet i la seva configuració
sudo apt autoremove       # elimina dependències innecessàries
sudo apt search paraula   # busca paquets
apt show paquet           # mostra informació d'un paquet

```

La diferència clau entre `apt update` i `apt upgrade` és que el primer només actualitza la informació sobre quins paquets hi ha disponibles, i el segon descarrega i instal·la les noves versions. Sempre s'ha de fer `update` abans de `upgrade`, i és bo fer-los junts.

Quan s'instal·la un paquet, `apt` resol automàticament les **dependències**: si volem instal·lar un programa que necessita una biblioteca, `apt` la descarrega i la instal·la sense que l'hi demanem. Aquesta és una de les grans virtuts de Debian.

3.5 L'editor nano

Per editar fitxers de configuració, necessitem un editor de text en consola. N'hi ha molts: **nano**, **vim**, **emacs**, **micro**, **joe**. Al BernatLab farem servir **nano**, que és el més amable per a principiants i que ve preinstal·lat a Debian Lite.

Per obrir-lo:

```
nano /etc/sudoers
```

A la part inferior de la pantalla veuràs les dreceres de teclat. Les més útils:

- `Ctrl+O` → desar (write Out)
- `Ctrl+X` → sortir
- `Ctrl+K` → tallar una línia
- `Ctrl+U` → enganxar
- `Ctrl+W` → buscar
- `Ctrl+G` → ajuda

Important: nano per sí sol no demana confirmació abans de desar. Polsar `Ctrl+X` i respondre `Y` desarà i sortirà. `Ctrl+C` cancel·la.

Si volem un editor una mica més potent, podem instal·lar **micro** (`sudo apt install micro`) o aprendre **vim** (un viatge al·lucinant, però que val la pena si volem ser administradors de veritat).

3.6 Serveis i systemd

Ja hem parlat de systemd al Capítol 2. Aquí aprofundim en la gestió quotidiana dels serveis.

```

systemctl status servei    # estat detallat
systemctl start servei     # iniciar
systemctl stop servei      # aturar
systemctl restart servei   # reiniciar
systemctl reload servei    # recarregar config sense parar
systemctl enable servei    # arrencar a l'inici

```

```
systemctl disable servei      # no arrencar a l'inici
systemctl is-active servei    # actiu? (yes/no)
systemctl is-enabled servei   # habilitat? (yes/no)
systemctl list-units --type=service --state=running
systemctl list-unit-files --type=service
```

A Debian, la majoria de serveis tenen el seu fitxer de configuració a `/etc/nomservei/`. Per exemple, la configuració de Docker és a `/etc/docker/`, i la de SSH a `/etc/ssh/sshd_config`.

Per veure els logs d'un servei:

```
journalctl -u servei          # tots els logs
journalctl -u servei -f       # en directe (follow)
journalctl -u servei --since "1 hour ago"
journalctl -u servei --since today
```

`journalctl` és l'eina estàndard de `systemd` per accedir als logs. Combina tots els missatges de tots els serveis en una base de dades indexada, i permet filtres potents per data, prioritat, unitat, etc.

3.7 Processos

Cada programa que s'executa al sistema és un **procés**, amb un identificador únic (**PID**). Per veure'ls:

```
ps aux                        # tots els processos
ps aux | grep docker         # processos que contenen "docker"
top                           # vista dinàmica, ordenada per ús
htop                          # millor que top, cal instal·lar-lo
```

Si volem instal·lar `htop` (recomanable):

```
sudo apt install htop
```

Per matar un procés:

```
kill PID                      # envia SIGTERM (amable)
kill -9 PID                   # envia SIGKILL (forçat)
killall nom                   # mata tots els processos amb aquest nom
pkill -f patró                # mata processos que coincideixen amb un patró
```

Quan un contenidor Docker no respon, sovint el "matarem" amb `docker kill` (que veurem al Capítol 5), però entendre `kill` i `ps` és fonamental.

3.8 Logs

Els logs són la memòria del sistema. Quan alguna cosa falla, els logs són la primera — i sovint l'única — pista que tenim per entendre per què.

Els principals fitxers de log a Debian són:

- `/var/log/syslog` → log general del sistema
- `/var/log/auth.log` → autenticacions (SSH, sudo)
- `/var/log/kern.log` → missatges del kernel
- `/var/log/mail.log` → correu
- `/var/log/docker/` → logs de Docker
- `/var/log/apt/` → historial d'instal·lacions apt

Comandes útils:

```
tail -f /var/log/syslog
less /var/log/syslog
grep "error" /var/log/syslog
journalctl -xe           # últims missatges amb explicacions
journalctl --since "1 hour ago"
```

Al BernatLab, els logs poden créixer molt. Una bona pràctica és configurar **logrotate** (que ja ve amb Debian) perquè els vagi rotant i comprimint, i **log2ram** per moure'ls a RAM, reduint les escriptures a la targeta microSD.

3.9 La carpeta /home/bernat/homelab

Aquí és on viu la nostra feina. L'estructura recomanada per al BernatLab és:

```
/home/bernat/homelab/
■■■■ README.md           # descripció general
■■■■ CHANGELOG.md        # registre de canvis
■■■■ docker-compose.yml  # definició principal de serveis
■■■■ .env                 # variables d'entorn (no versionat)
■■■■ .gitignore           # què no versionar
■■■■ stacks/              # sub-piles de compose
■   ■■■■ monitoring/      # Uptime Kuma, Grafana, etc.
■   ■■■■ data/            # InfluxDB, PostgreSQL, etc.
■   ■■■■ iot/             # Mosquitto, Node-RED, etc.
■   ■■■■ media/           # File Browser, foto, música
■■■■ data/                # volums persistents
■   ■■■■ uptime-kuma/
■   ■■■■ homepage/
■   ■■■■ portainer/
■   ■■■■ ...
■■■■ backup/              # còpies de seguretat
■■■■ scripts/             # scripts de manteniment
■■■■ docs/                # documentació addicional
```

Aquesta estructura ens permet:

- **Tenir un sol fitxer `docker-compose.yml`** a l'arrel, o bé dividir-lo en piles temàtiques.
- **Guardar les dades persistents** dins de `data/`, cosa que facilita les còpies de seguretat.
- **Versionar tot** amb Git, excepte `.env` i `data/` (gràcies a `.gitignore`).
- **Documentar** dins del projecte, no en un altre lloc.

3.10 Comandes útils del dia a dia

Aquesta és la llista que consultarem cada setmana:

```
# Sistema
uname -a           # kernel i arquitectura
uptime            # temps encès, càrrega
date              # data i hora
who               # qui està connectat
w                 # qui i què fa

# Disc
df -h             # espai en sistemes de fitxers
du -sh /camí/     # espai ocupat per un directori
lsblk             # llista de dispositius de bloc

# Xarxa
```

```
ip a          # adreces IP
ip r          # rutes
ss -tulpn     # ports oberts
ping host     # comprovar connectivitat

# Processos
ps aux
htop
pkill -f patro

# Logs
journalctl -xe
tail -f /var/log/syslog

# Serveis
systemctl status servei
systemctl restart servei
```

3.11 Bones pràctiques

1. **Mai no treballis com a root per defecte.** Usa un usuari normal i puja amb `sudo` quan calgui.
2. **Documenta cada canvi important.** Un cop fet, un parell de línies al `CHANGELOG.md`.
3. **Fes còpies de seguretat abans de canvis grans.** `apt full-upgrade`, canvis de configuració, actualitzacions de contenidors. Si alguna cosa es trenca, has de poder tornar enrere.
4. **Mira els logs quan alguna cosa falla.** No reinicis a cegues. Llegeix, entén, actua.
5. **No instal·lis programes que no necessitis.** Cada programa és un risc potencial i un consumidor de recursos.
6. **Aprèn una ordre abans de memoritzar el seu àlies.** Els àlies estan bé, però entén el que fas servir.

3.12 Errors habituals

Error 1: executar ordres com a root "perquè sí". Síntoma: fitxers propietat de root que després no podem editar, o canvis irreversibles. Solució: usar `sudo` només quan és estrictament necessari.

****Error 2: `rm -rf` sense comprovar**.** Síntoma: hem esborrat alguna cosa que no tocava. Solució: llegir dues vegades abans de prémer Enter. Mai no executar `rm -rf /` o `rm -rf *` en un directori arrel.

Error 3: instal·lar paquets innecessaris. Síntoma: sistema ple de coses, ports oberts, serveis innecessaris. Solució: abans d'instal·lar, preguntar-se si realment cal.

Error 4: no mirar els logs. Síntoma: quan alguna cosa falla, no sabem per què. Solució: `journalctl -xe` i `tail -f` són els nostres millors amics.

****Error 5: editar `/etc/sudoers` a mà amb nano**.** Síntoma: `sudo` es trenca i no podem arreglar res. Solució: usar sempre `visudo`, que valida la sintaxi abans de desar.

3.13 Esquema del sistema

Esquema Mermaid (text):

```

graph TB
    subgraph Usuari["Espai d'usuari"]
        SHELL["Shell (bash)"]
        TOOLS["Eines: nano, htop, less, grep"]
    end

    subgraph Sistema["Sistema operatiu"]
        KERNEL["Kernel Linux<br/>(gestió de recursos)"]
        SYSTEMD["systemd<br/>(gestor de serveis)"]
        LOGS["journald + /var/log"]
    end

    subgraph Disc["Sistema de fitxers"]
        EXT4["/ (root)"]
        DATA["/home/bernat/homelab"]
        ETCC["/etc (configuració)"]
    end

    subgraph Hard["Maquinari"]
        CPU["CPU ARM"]
        RAM["RAM 4 GB"]
        SD["microSD"]
    end

    SHELL --> TOOLS
    TOOLS --> KERNEL
    SYSTEMD --> KERNEL
    LOGS --> KERNEL
    KERNEL --> CPU
    KERNEL --> RAM
    KERNEL --> SD
    EXT4 --> SD
    DATA --> EXT4
    ETCC --> EXT4

```

3.14 Resum

Hem après les ordres bàsiques de Linux aplicades al BernatLab: sistema de fitxers, usuaris, permisos, sudo, paquets apt, nano, serveis amb systemd, processos, logs. També hem vist com ha d'estar organitzada la carpeta `/home/bernat/homelab` i quines bones pràctiques seguir. En el proper capítol pujarem de nivell: xarxa, SSH i Tailscale.

3.15 Exercicis pràctics

1. Entra a `/home/bernat/homelab` i comprova què hi ha. Si no existeix, crea-la: `mkdir -p /home/bernat/homelab/{stacks,data,backup,scripts,docs}`.
2. Executa `ls -la /home/bernat/homelab` i explica què hi veus.
3. Mira l'ús de disc amb `df -h` i l'ús de memòria amb `free -h`. Anota els valors.
4. Executa `systemctl list-units --type=service --state=running` i compta quants serveis hi ha actius.
5. Mira les últimes 20 línies de `/var/log/syslog` amb `tail -n 20 /var/log/syslog`. Hi ha algun error o advertència?
6. Instal·la `htop` amb `sudo apt install htop`, executa'l i surt amb `q`.

Comandes útils del capítol:

```

pwd, ls, cd, cp, mv, rm, mkdir
chmod, chown, sudo
apt update && apt upgrade
apt install, apt remove

```



```
nano, systemctl, journalctl  
ps aux, htop, kill  
df -h, du -sh, free -h  
ip a, ss -tulpn
```

Paraules clau: **filesystem, FHS, permisos, sudo, apt, nano, systemd, journalctl, htop, /var/log, homelab, bones pràctiques.**

Capítol 4 — Xarxa, SSH i Tailscale

"Un servidor sense xarxa és una caixa forta en una illa deserta. Un servidor amb bona xarxa és una extensió de tu mateix."

4.1 Què vol dir "xarxa" en un servidor

Quan diem "xarxa" en el context d'un servidor, ens referim a la capacitat de la màquina de comunicar-se amb altres dispositius. Aquesta comunicació es fa a través de **protocols** (com TCP/IP), a través de **ports** (com el 22 per a SSH, el 80 per a HTTP, el 443 per a HTTPS), i a través d'**adreces** que identifiquen cada dispositiu de forma única.

Al BernatLab, la xarxa és fonamental perquè:

- Accedim al servidor remotament per SSH.
- Els serveis (Portainer, Uptime Kuma, Homepage) escolten peticions en ports específics.
- La Raspberry es comunica amb sensors, amb serveis al núvol, amb la base de dades.
- Volem accedir-hi des de qualsevol lloc del món — des de la feina, des del mòbil, des d'un portàtil de viatge.

Aquest capítol explica els conceptes bàsics i, sobretot, com fer que tot això funcioni **de manera segura** amb Tailscale.

4.2 IP, ports i DNS

Adreces IP

Una adreça **IP** (Internet Protocol) és un número que identifica un dispositiu en una xarxa. En la versió 4 (IPv4), és un número de 32 bits que s'escriu com quatre grups de fins a tres xifres separats per punts, p. ex. `192.168.1.42` o `100.115.134.76`.

A la nostra Raspberry podem veure les adreces assignades amb:

```
ip a
```

Aquesta ordre ens mostrarà diverses interfícies de xarxa:

- **lo** (loopback): la interfície "cap a un mateix", amb adreça `127.0.0.1`. Sempre present.
- **eth0**: la interfície Ethernet, per cable. Aquesta serà la principal al BernatLab.
- **wlan0** (si l'habilitem): la interfície Wi-Fi.

En una xarxa domèstica, la Raspberry tindrà una adreça del tipus `192.168.x.y`, assignada pel router (que fa de servidor DHCP). Aquesta és la **IP privada local**: només és visible des de dins de casa.

Quan la Raspberry parla amb Internet, el router tradueix la seva IP privada per la **IP pública** que ens ha assignat l'operador. Això es fa gràcies al **NAT** (Network Address Translation). El problema: aquesta IP pública canvia de tant en tant (llevat que paguem una IP fixa), i des de fora de casa no sabem quina és en cada moment.

Ports

Un **port** és un número de 16 bits (de 0 a 65535) que identifica una aplicació concreta dins d'una màquina. Quan un programa vol rebre connexions, escolta en un port. Per convenció:

- **22**: SSH
- **80**: HTTP
- **443**: HTTPS
- **3000-3999**: aplicacions web (Homepage, Grafana)
- **3001**: Uptime Kuma
- **9443**: Portainer
- **5432**: PostgreSQL
- **1883**: MQTT

Quan accedim a `100.115.134.76:9443`, estem dient: "vull connectar a la IP `100.115.134.76`, port `9443`". El port és part essencial de l'adreça.

Per veure quins ports té oberts la nostra màquina:

```
ss -tulpn
```

Aquesta ordre ens mostrarà totes les connexions TCP/UDP obertes, amb el protocol, l'adreça local, el port i el procés que les manté. És una eina molt útil per entendre què està passant.

DNS

El **DNS** (Domain Name System) és el sistema que tradueix noms fàcils de recordar (`bernatmora.github.io`) en adreces IP (`140.82.121.4` o similar). Quan posem una adreça al navegador, el sistema consulta un servidor DNS per obtenir la IP.

A Debian, la configuració de DNS és a `/etc/resolv.conf`. Normalment hi veurem:

```
nameserver 192.168.1.1
```

que apunta al router, que al seu sap on són els servidors DNS de l'operador (o pot ser ell mateix, fent de DNS forwarder).

4.3 Tallafof i seguretat bàsica

Un **tallafof** (firewall) és un programa que decideix quin tràfic pot entrar i sortir d'una màquina. A Debian, el tallafof estàndard és **nftables** (l'evolució de l'antic iptables), tot i que molts encara prefereixen la interfície **ufw** (Uncomplicated Firewall) per la seva senzillesa.

A l'instal·lació de Debian Lite, el tallafof pot estar totalment obert (acceptant tot el tràfic) o parcialment tancat, depenent de la configuració. Per veure l'estat:

```
sudo ufw status
```

Si **ufw** no està instal·lat:

```
sudo apt install ufw
```

Regles bàsiques que podem aplicar al BernatLab:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow 9443/tcp      # Portainer
sudo ufw allow 3001/tcp      # Uptime Kuma
sudo ufw allow 3000/tcp      # Homepage
sudo ufw enable
```

Això obre els ports estrictament necessaris i tanca la resta. A la pràctica, com que Tailscale ja ens dona una xarxa privada i el router ja ens protegeix, el tallafoc del servidor és una **bona capa addicional** però no l'única.

4.4 SSH: la porta d'entrada

SSH (Secure Shell) és el protocol estàndard per accedir a una consola remota de forma segura. Quan obrim una sessió SSH, tot el que escrivim i tot el que rebem viatja **xifrat** per la xarxa, de manera que ningú no el pot interceptar.

Al BernatLab, SSH és la nostra eina principal. Treballem habitualment des del nostre PC o portàtil, connectant-nos a la Raspberry per SSH. Sense SSH, hauríem de tenir un monitor, un teclat i un ratolí connectats a la Raspberry — inviable per a un servidor 24/7.

Com funciona SSH

Quan un client SSH es connecta a un servidor, passa el següent:

1. **Handshake inicial:** el client i el servidor intercanvien informació sobre les versions del protocol i quins algorismes de xifratge suporten.
2. **Intercanvi de claus:** s'estableix una connexió xifrada mitjançant un sistema asimètric (claus públiques i privades).
3. **Autenticació:** el client demostra la seva identitat al servidor. Pot fer-ho amb: - **Contrasenya:** el mètode més senzill però menys segur. - **Clau pública:** el mètode recomanable, basat en criptografia asimètrica.
4. **Sessió establerta:** a partir d'aquí, tota la comunicació viatja xifrada.

Connexió bàsica

```
ssh bernat@100.115.134.76
```

Això connecta a la IP **100.115.134.76** (la Tailscale), com a usuari **bernat**. Ens demanarà la contrasenya (o la clau, si està configurada) i, un cop autenticats, estarem dins de la consola del servidor.

Claus SSH: autenticació sense contrasenya

El mètode recomanat per a SSH és l'**autenticació amb clau pública**. En lloc d'escriure una contrasenya cada vegada, generem un parell de claus:

- **Clau privada:** queda al nostre PC (al fitxer `~/.ssh/id_ed25519`). **No l'hem de compartir mai.**
- **Clau pública:** la pengem al servidor, al fitxer `~/.ssh/authorized_keys`. Qualsevol pot veure-la — és pública.

Quan ens connectem, el client demostra al servidor que posseeix la clau privada sense enviar-la mai per la xarxa. Això és molt més segur que una contrasenya, perquè:

- Una clau de 256 bits és pràcticament impossible d'endevinar.
- La clau privada no viatja mai per la xarxa.
- Podem protegir la clau amb una passphrase (contrasenya local).

Per generar el parell de claus al nostre PC client:

```
ssh-keygen -t ed25519 -C "bernat@bernatlab"
```

Ens demanarà una passphrase (recomanable) i guardarà les claus a `~/.ssh/id_ed25519` (privada) i `~/.ssh/id_ed25519.pub` (pública).

Per pujar la clau pública al servidor:

```
ssh-copy-id bernat@100.115.134.76
```

A partir d'ara, podem entrar sense contrasenya (o amb la passphrase de la clau). El servidor confia en nosaltres perquè tenim la clau privada que correspon a la clau pública que ha acceptat.

Configuració del servidor SSH

L'arxiu principal és `/etc/ssh/sshd_config`. Algunes directives recomanables:

```
Port 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers bernat
```

- **PermitRootLogin no:** impedeix que l'usuari root entri directament. Cal entrar com a `bernat` i fer `sudo`.
- **PasswordAuthentication no:** desactiva l'autenticació per contrasenya. Només es pot entrar amb clau.
- **AllowUsers bernat:** només l'usuari `bernat` pot entrar per SSH.

Després de modificar el fitxer, cal reiniciar el servei:

```
sudo systemctl restart ssh
```

I, IMPORTANT, abans de tancar la sessió, comprovar que podem reconnectar. Si hem deshabilitat l'accés per contrasenya sense pujar cap clau, ens quedarem fora del servidor.

SSH segur: bones pràctiques

1. **Usa claus, no contrasenyes.**
2. **Desactiva l'accés de root.**
3. **Limita els usuaris** que poden entrar.
4. **Considera canviar el port 22 per un altre** (per dissuadir els bots que escanegen la xarxa). No és seguritat real, però redueix el soroll als logs.
5. **Mantén el servidor actualitzat:** `sudo apt update && sudo apt upgrade`.
6. **Monitoritza els intents de connexió** amb `journalctl -u ssh` o eines com `fail2ban`.

4.5 Tailscale: la xarxa privada

Aquí és on el BernatLab esdevé interessant. **Tailscale** és una eina que ens permet crear una **xarxa privada virtual (VPN)** entre tots els nostres dispositius, sense tocar el router, sense obrir ports, sense configurar NAT.

Què és exactament

Tailscale es basa en **WireGuard**, un protocol de VPN modern, ràpid i segur. La diferència amb una VPN tradicional és que Tailscale s'encarrega de tota la complexitat: trobar els dispositius a través de NAT, travessar tallafocs, gestionar les claus. Nosaltres només hem d'instal·lar el client, autenticar-nos amb un compte, i tots els nostres dispositius apareixen en una xarxa comuna.

La nostra xarxa

Al BernatLab tenim:

- La Raspberry Pi (hostname `hortosona`) → IP Tailscale `100.115.134.76`
- El PC amb Windows on treballem (BernatMora) → una altra IP `100.x.y.z`
- El mòbil Android, si hi instal·lem Tailscale → una altra IP `100.x.y.z`
- Potser un servidor a casa dels pares, una màquina virtual, etc.

Tots aquests dispositius es poden veure entre ells, com si estiguessin connectats a un switch virtual invisible. I tot, xifrat de punta a punta.

Com s'ha instal·lat

La instal·lació a Debian és directa:

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

Aquesta segona ordre ens dona un enllaç per autenticar-nos al navegador. Un cop autèntiquem el compte, la màquina apareix a la nostra xarxa Tailscale i li assigna una IP del rang `100.x.y.z`.

Per veure l'estat:

```
tailscale status
```

Per veure la nostra IP:

```
tailscale ip -4
```

MagicDNS: noms en lloc d'IPs

Una de les funcions més útils de Tailscale és **MagicDNS**. Un cop activat, podem accedir a qualsevol dispositiu de la nostra xarxa pel seu nom, sense recordar la IP. Per exemple:

```
ssh bernat@hortosona
```

en lloc de:

```
ssh bernat@100.115.134.76
```

Això funciona perquè Tailscale munta un servidor DNS local (`100.100.100.100`) que resol els noms dels nostres dispositius. Si al nostre PC client tenim Tailscale instal·lat i actiu, podem accedir a `http://hortosona:3000` en lloc de `http://100.115.134.76:3000`. Més net, més fàcil de

recordar.

Per què és tan important

Tres raons:

1. **Seguretat**: cap port de la Raspberry està exposat a Internet. Només els dispositius de la nostra xarxa Tailscale hi poden accedir. Això és una capa de seguretat brutalment efectiva.
2. **Simplicitat**: no hem de configurar NAT, no hem d'obrir ports al router, no hem de gestionar IP dinàmica. Tailscale ho fa tot.
3. **Mobilitat**: podem accedir al BernatLab des d'una Wi-Fi pública, des del 4G del mòbil, des d'un cafè amb WiFi. La xarxa Tailscale funciona igual a tot arreu.

Taildrop: compartició de fitxers

Tailscale inclou **Taildrop**, una eina per enviar fitxers entre dispositius de la xarxa. És útil, per exemple, per pujar una captura de pantalla del mòbil a la Raspberry, o per descarregar una còpia de seguretat al PC.

Compartició de serveis amb altres

Si volem que algú altre (un amic, un company de feina) accedeixi a un dels nostres serveis sense tenir Tailscale, podem fer-ho amb **Tailnet Share** o, més senzill, exposant un port específic amb `tailscale serve`. Però de moment, al BernatLab, tots els serveis són d'ús personal.

4.6 Sortir a Internet: com es connecta la Raspberry a la xarxa

Fins aquí hem parlat de com la Raspberry escolta peticions entrants. Però, evidentment, la Raspberry també ha de poder parlar amb l'exterior: per actualitzar paquets amb `apt`, per descarregar imatges Docker, per fer pings als serveis que monitoritzem.

Això ho fa gràcies a la configuració de xarxa per defecte: la interfície `eth0` (o `wlan0`) rep una IP del router via DHCP, el router li dona accés a Internet a través del NAT, i el sistema configura el gateway per defecte. Per veure la ruta per defecte:

```
ip r
```

que ens mostrarà una línia tipus:

```
default via 192.168.1.1 dev eth0
```

que vol dir: "tot el tràfic que no és per a la xarxa local, passa pel router `192.168.1.1` per la interfície `eth0`".

Els servidors DNS estan configurats a `/etc/resolv.conf`, que sol apuntar al router o a servidors com `8.8.8.8` (Google) o `1.1.1.1` (Cloudflare).

4.7 Esquema de xarxa del BernatLab

Esquema Mermaid (text):

```
graph TB
    subgraph Casa["Xarxa local de casa (192.168.x)"]
        RPI["Raspberry Pi<br/>hortosona<br/>192.168.1.x"]
        ROUTER["Router<br/>192.168.1.1"]
    end
```

```

end

subgraph Tailscale["Xarxa Tailscale (100.x)"]
    TS_RPI["RPI → 100.115.134.76"]
    TS_PC["PC Bernat → 100.x.y.z"]
    TS_MOB["Mòbil → 100.x.y.z"]
end

subgraph Internet["Internet"]
    APT["Repositoris apt"]
    DOCKER["Docker Hub"]
    GH["GitHub"]
    WEB["Hort Osona web"]
end

RPI <--> ROUTER
ROUTER <--> Internet
RPI -.->|Tailscale| TS_RPI
PC["PC Bernat"] -.->|Tailscale| TS_PC
MOB["Mòbil"] -.->|Tailscale| TS_MOB
TS_RPI <--> TS_PC
TS_RPI <--> TS_MOB

style Casa fill:#e8f5e9
style Tailscale fill:#e3f2fd
style Internet fill:#fff3e0

```

4.8 Comandes útils

```

# Adreça IP
ip a
ip r
hostname -I

# DNS
cat /etc/resolv.conf
nslookup bernatmora.github.io
dig hortosona

# Ports
ss -tulpn
netstat -tulpn      # alternativa antiga

# SSH
ssh bernat@hortosona
ssh -i ~/.ssh/id_ed25519 bernat@100.115.134.76
scp fitxer.txt bernat@hortosona:~/

# Tailscale
tailscale status
tailscale ip -4
tailscale ping hortosona

```

4.9 Errors habituals

Error 1: entrar per SSH amb contrasenya en lloc de clau. Síntoma: cada vegada que entrem, hem d'escriure la contrasenya. Solució: generar clau i penjar-la al servidor.

Error 2: deshabilitar l'autenticació per contrasenya sense pujar cap clau. Síntoma: no podem entrar al servidor. Solució: connectar-hi un monitor i teclat, editar `/etc/ssh/sshd_config`, reiniciar.

Error 3: pensar que Tailscale ens protegeix de tot. Síntoma: ens relaxem massa. Solució: Tailscale és una capa, no l'única. Calen contrasenyes fortes, clau de SSH, tallafoc, actualitzacions.

Error 4: obrir ports al router de casa. Síntoma: el router queda exposat a Internet. Solució: mai no cal amb Tailscale. Si algú t'ho suggereix, desconfia.

Error 5: no documentar les claus SSH. Síntoma: perdem una clau, no podem entrar. Solució: desar una còpia segura de la clau privada (en un gestor de contrasenyes o en un dispositiu segur).

4.10 Resum

Hem après què és una IP, un port, un DNS, un tallafoc. Hem après a configurar SSH amb claus públiques, a deshabilitar l'accés per contrasenya, a limitar els usuaris. Hem après què és Tailscale, com funciona, com ens dóna una xarxa privada sense tocar el router, i com MagicDNS ens permet accedir als serveis pel nom. Ara ja podem connectar-nos al BernatLab de manera segura des de qualsevol lloc del món. En el proper capítol començarem a desplegar-hi serveis amb Docker.

4.11 Exercicis pràctics

1. Comprova la teva IP Tailscale amb `tailscale ip -4`.
2. Comprova l'estat dels dispositius de la teva xarxa amb `tailscale status`.
3. Connecta't a la Raspberry per SSH usant el nom en lloc de la IP: `ssh bernat@hortosona`.
4. Mira els últims accessos SSH amb `journalctl -u ssh --since "1 day ago"`.
5. Comprova quins ports tens oberts amb `ss -tulpn`. Quants serveis escolten? Quins?
6. Comprova si tens el tallafoc actiu: `sudo ufw status`. Si no el tens, instal·la'l i configura'l com s'ha explicat.
7. Genera una clau SSH al teu PC (si no en tens) i puja-la al servidor. Comprova que pots entrar sense contrasenya.

Comandes útils:

```
ip a, ip r, ss -tulpn
ssh bernat@hortosona
ssh-keygen -t ed25519
ssh-copy-id bernat@hortosona
sudo ufw status
sudo ufw allow ssh
tailscale status
tailscale ip -4
```

Paraules clau: **IP, port, DNS, tallafoc, SSH, clau pública, WireGuard, Tailscale, MagicDNS, NAT, VPN, seguretat.**

Capítol 5 — Docker des de zero

*“Docker no és una eina. Docker és una manera de pensar els serveis: cadascun en la seva caixa, tots parlant entre ells, cap contaminant l'altre.”**

5.1 Què és Docker i per què serveix

Docker és una eina que ens permet executar aplicacions dins de **contenidors**: entorns aïllats, lleugers, portables, que contenen tot el que una aplicació necessita per funcionar. Un contenidor és, en essència, un **procés del sistema operatiu** que ve amb el seu propi sistema de fitxers, la seva pròpia interfície de xarxa i el seu propi cicle de vida — però sense portar un sistema operatiu sencer com ho faria una màquina virtual.

La diferència amb una màquina virtual és clau. Una VM emula un maquinari complet i executa un sistema operatiu sencer dins: pèrdues de rendiment evidents, consum de RAM important, arrencada lenta. Un contenidor comparteix el kernel de la màquina amfitriona (la nostra Raspberry) i només porta les biblioteques, configuracions i binaris que l'aplicació necessita. Com a resultat:

- Arrenca en segons, no en minuts.
- Consumeix pocs megabytes de RAM, no gigabytes.
- És totalment portable: un contenidor que funciona a la Raspberry, funcionarà a un servidor professional, al PC, a un núvol.

Docker ens permet tractar cada servei del BernatLab com una **unitat independent i autosuficient**. Portainer és un contenidor. Uptime Kuma és un altre. Homepage és un altre. Si un falla, els altres continuen funcionant. Si en volem actualitzar un, els altres ni se n'assabenten. Si volem moure'ns a un altre servidor, podem emportar-nos els contenidors com si fossin caixes de cartró ben etiquetades.

5.2 Conceptes fonamentals

Per entendre Docker, cal dominar quatre conceptes:

Imatge

Una **imatge** Docker és un fitxer (o un conjunt de fitxers) que conté tot el necessari per executar un servei: el codi, les biblioteques, les configuracions per defecte, les dependències del sistema. Pensa en una imatge com una **plantilla de només lectura**: un motlle a partir del qual crearem contenidors.

Les imatges es distribueixen des de registres, el més conegut dels quals és **Docker Hub** (hub.docker.com). Quan fem `docker pull portainer/portainer-ce:latest`, estem dient: "descarrega la imatge oficial de Portainer, en la seva última versió".

Les imatges es construeixen amb un fitxer anomenat **Dockerfile**, que és una recepta: "comença amb una imatge base, instal·la aquestes dependències, copia aquest codi, exposa aquest port, executa aquesta ordre". Però al BernatLab no construirem imatges pròpies; usarem les que la comunitat ja ha publicat.

Contenidor

Un **contenidor** és una instància viva d'una imatge. Si la imatge és la plantilla, el contenidor és la instància en execució. Podem tenir diversos contenidors de la mateixa imatge (per exemple, tres instàncies de Redis fent papers diferents) cadascun amb el seu propi estat.

Quan executem un contenidor, Docker hi assigna un identificador únic (un hash de 64 caràcters, encara que normalment en veiem només els 12 primers) i un nom aleatori (com `portainer-app-1` o `agitated_mclean`). Podem donar-li un nom nosaltres amb `--name`.

Volum

Un **volum** és un emmagatzematge persistent, gestionat per Docker, que viu fora del sistema de fitxers del contenidor. Per què és important? Perquè els contenidors, per naturalesa, són **efímers**: quan els esborrem, tot el seu sistema de fitxers desapareix. Si el contenidor de Portainer té la seva configuració dins, i l'esborrem per accident, perdem la configuració. Per això, tot allò que volem conservar — bases de dades, configuracions, fitxers pujats — es munta en un volum.

Hi ha dos tipus de volums:

- **Volums anomenats** (named volumes): gestionats per Docker, amb noms com `portainer_data`. Es guarden a `/var/lib/docker/volumes/`.
- **Muntatges de bind** (bind mounts): apuntem a una carpeta concreta del sistema amfitrió, com `/home/bernat/homelab/data/portainer`. Al BernatLab farem servir aquesta segona opció, perquè ens permet tenir les dades dins de la nostra carpeta de treball, fàcil de copiar i versionar.

Xarxa

Una **xarxa** Docker és un espai aïlat on els contenidors es poden comunicar entre ells pel nom. Per defecte, Docker crea una xarxa **bridge** que comunica tots els contenidors entre ells. Podem crear xarxes personalitzades per organitzar-los.

Això ens permet fer coses com: el contenidor de Uptime Kuma pot parlar amb el de Portainer pel nom `portainer`, sense necessitat d'IP ni de ports exposats a l'exterior.

5.3 Instal·lació i verificació

A Debian 13, Docker es pot instal·lar des dels repositoris oficials de Docker (no pas de Debian, perquè la versió empaquetada per Debian acostuma a ser antiga). La seqüència estàndard és:

```
# Eliminar versions antigues
sudo apt remove docker docker-engine docker.io containerd runc

# Afegir el repositori oficial
sudo apt install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian gpgkey" | sudo tee /etc/apt/sources.list.d/docker.list >/dev/null

# Instal·lar
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Afegir l'usuari bernat al grup docker
sudo usermod -aG docker bernat
```

L'última ordre és fonamental: sense ella, només `root` pot executar ordres Docker. En afegir `bernat` al grup `docker`, podem fer `docker ps` sense `sudo`. Perquè el canvi tingui efecte, cal tancar i tornar a obrir la sessió SSH.

Per comprovar que tot funciona:

```
docker run hello-world
```

Aquesta ordre descarrega una imatge mínima, executa un contenidor que imprimeix un missatge de benvinguda, i surt. Si veiem el missatge, Docker està correctament instal·lat.

5.4 Ordres essencials

Aquestes són les ordres que farem servir cada dia. Convé memoritzar-les o, si més no, tenir-les a mà.

Imatges

<code>docker images</code>	<code># llista imatges locals</code>
<code>docker pull imatge:tag</code>	<code># descarrega una imatge</code>
<code>docker rmi imatge</code>	<code># esborra una imatge</code>
<code>docker image prune</code>	<code># esborra imatges no usades</code>
<code>docker image prune -a</code>	<code># esborra TOTES les imatges no usades</code>

Contenidors

<code>docker ps</code>	<code># contenidors actius</code>
<code>docker ps -a</code>	<code># tots els contenidors (inclosos aturats)</code>
<code>docker run -d --name web nginx</code>	<code># crea i arrenca un contenidor</code>
<code>docker start nom</code>	<code># arrenca un contenidor existent</code>
<code>docker stop nom</code>	<code># atura un contenidor</code>
<code>docker restart nom</code>	<code># reinicia</code>
<code>docker rm nom</code>	<code># esborra un contenidor aturat</code>
<code>docker rm -f nom</code>	<code># forçar esborrat</code>
<code>docker logs nom</code>	<code># mostra logs</code>
<code>docker logs -f nom</code>	<code># logs en directe</code>
<code>docker exec -it nom bash</code>	<code># obre una consola dins del contenidor</code>
<code>docker stats</code>	<code># ús de recursos en temps real</code>

Volums

<code>docker volume ls</code>	<code># llista volums</code>
<code>docker volume create nom</code>	<code># crea un volum</code>
<code>docker volume rm nom</code>	<code># esborra un volum</code>
<code>docker volume prune</code>	<code># esborra volums no usats</code>
<code>docker volume inspect nom</code>	<code># info d'un volum</code>

Xarxes

<code>docker network ls</code>	<code># llista xarxes</code>
<code>docker network create nom</code>	<code># crea una xarxa</code>
<code>docker network rm nom</code>	<code># esborra una xarxa</code>
<code>docker network inspect nom</code>	<code># info d'una xarxa</code>

Sistema

<code>docker system df</code>	<code># espai ocupat</code>
<code>docker system prune</code>	<code># neteja general</code>
<code>docker system prune -a --volumes</code>	<code># neteja total (compte!)</code>
<code>docker info</code>	<code># informació del sistema Docker</code>

5.5 Exemple pràctic: Nginx en un contenidor

Per veure-ho en acció, despleguem Nginx (un servidor web lleuger) en un contenidor. Això ens permetrà entendre tot el mecanisme sense complicacions:

```
docker run -d --name web -p 8080:80 nginx
```

Desglossant:

- **-d**: detached, en segon terme. Si no, la consola es queda penjada.
- **--name web**: li donem un nom.
- **-p 8080:80**: mapegem el port 80 del contenidor al port 8080 de la Raspberry.
- **nginx**: la imatge.

Ara podem obrir un navegador i anar a <http://100.115.134.76:8080>. Veurem la pàgina de benvinguda de Nginx.

Quan volguem netejar:

```
docker stop web
docker rm web
```

Ara, el mateix amb **persistència** (volum) i **configuració**:

```
mkdir -p /home/bernat/homelab/data/nginx
docker run -d \
  --name web \
  -p 8080:80 \
  -v /home/bernat/homelab/data/nginx:/usr/share/nginx/html:ro \
  nginx
```

Amb **-v** hem muntat una carpeta local dins del contenidor, en mode només lectura (**ro**). Si creem un fitxer **index.html** a **/home/bernat/homelab/data/nginx/**, el Nginx el servirà. Si esborrem el contenidor, les dades continuen a la carpeta.

5.6 Per què Docker Compose i no docker run

Fins aquí hem vist **docker run**, que és perfectament vàlid per a un contenidor aïllat. Però al BernatLab tenim diversos serveis que volem gestionar junts, amb configuracions complexes, volums, xarxes, variables. Si els llancem un a un amb **docker run**, acabarem amb ordres llarguíssimes, impossibles de recordar, impossibles de reproduir.

Docker Compose ens permet descriure tot això en un sol fitxer **YAML**. Un cop definit, podem aixecar, parar, actualitzar tota la pila amb una sola ordre. I el fitxer YAML és **documentació en si mateix**: qualsevol que el llegeixi, sabrà exactament què s'està executant.

Exemple de fitxer **docker-compose.yml** per a Portainer:

```
services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: unless-stopped
    ports:
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

```
- /home/bernat/homelab/data/portainer:/data
```

Això és molt més llegible que la línia de `docker run` equivalent. I a mesura que afegim serveis, el fitxer creix ordenadament.

5.7 Docker Compose: estructura del fitxer

Un fitxer `docker-compose.yml` té tres seccions principals:

```
version: "3.8"    # versió de l'esquema (opcional en versions modernes)

services:         # definició dels serveis (contenidors)
  servei1:
    image: ...
    ports: ...
    volumes: ...
    environment: ...
    restart: ...
    depends_on:
      - altre_servei

volumes:          # definició de volums (opcional)
  volum1:

networks:         # definició de xarxes personalitzades (opcional)
  xarxa1:
```

Claus essencials per a cada servei

- **image:** quina imatge usar, amb tag opcional.
- **container_name:** nom del contenidor (opcional; per defecte, generat).
- **restart:** política de reinici. `unless-stopped` és la més comuna.
- **ports:** mapeig de ports `HOST:CONTENIDOR`.
- **volumes:** llista de volums o bind mounts.
- **environment:** variables d'entorn.
- **depends_on:** serveis dels quals depèn (ordre d'arrencada).
- **networks:** xarxes a les quals es connecta.

5.8 Ordres de Docker Compose

```
docker compose up -d      # aixeca tots els serveis
docker compose down       # atura i esborra contenidors
docker compose ps         # llista serveis
docker compose logs servei # logs d'un servei
docker compose logs -f    # logs en directe
docker compose restart servei # reinicia un servei
docker compose pull        # descarrega noves imatges
docker compose up -d       # re- crea contenidors amb noves imatges
docker compose exec servei bash # consola dins del contenidor
docker compose config      # valida el fitxer
```

Important: en versions modernes de Docker, l'ordre és `docker compose` (amb espai), no pas `docker -compose` (amb guió). La sintaxi antiga encara funciona si tenim l'eina instal·lada com a

binari, però la nova és la recomanada.

5.9 Exemple complet: Portainer + Uptime Kuma + Homepage

Aquest és el fitxer `docker-compose.yml` que tenim al BernatLab. Serveix d'exemple del que podem arribar a fer:

```
services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: unless-stopped
    ports:
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /home/bernat/homelab/data/portainer:/data

  uptime-kuma:
    image: louislam/uptime-kuma:latest
    container_name: uptime-kuma
    restart: unless-stopped
    ports:
      - "3001:3001"
    volumes:
      - /home/bernat/homelab/data/uptime-kuma:/app/data

  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - HOMEPAGE_ALLOWED_HOSTS=gethomepage.local:3000
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - /home/bernat/homelab/data/homepage:/app/config
      - /home/bernat/homelab/stacks/homepage:/app/config/widgets
```

Per aixecar tot això:

```
cd /home/bernat/homelab
docker compose up -d
```

I en qüestió de segons, els tres serveis estaran funcionant. Podem comprovar-ho amb `docker compose ps`.

5.10 Actualitzar contenidors

Una de les tasques habituals és mantenir els serveis actualitzats. El procediment estàndard és:

```
cd /home/bernat/homelab
docker compose pull          # descarrega noves versions
docker compose up -d        # re- crea els contenidors
docker image prune           # neteja imatges antigues
```

El `up -d` és intel·ligent: si la imatge ha canviat, Docker atura el contenidor antic, l'esborra i en crea un de nou amb la nova imatge. Si no ha canviat, no fa res.

Això es pot automatitzar amb **Watchtower** (un contenidor que vigila les actualitzacions) o amb **Diun** (que només avisa), o simplement amb un script al cron. Però abans d'automatitzar, fem-ho manualment unes quantes vegades per entendre què passa.

5.11 Volums i xarxes: com Docker els gestiona

Quan creem un contenidor amb un volum de bind mount, com `-v /home/bernat/homelab/data/portainer:/data`, el que fem és muntar una carpeta de l'amfitrió dins del contenidor. Els canvis que fem al contenidor es reflecteixen a l'amfitrió i viceversa. Si el contenidor s'esborra, la carpeta de l'amfitrió continua intacta.

Quan creem un contenidor amb un volum anomenat, com `-v portainer_data:/data`, Docker crea un directori a `/var/lib/docker/volumes/portainer_data/` i el munta dins del contenidor. La diferència pràctica: els volums anomenats són gestionats enterament per Docker, els bind mounts els gestionem nosaltres.

Al BernatLab, **preferim els bind mounts** perquè:

- Les dades són dins de la nostra carpeta de treball (`/home/bernat/homelab/data/`).
- Podem fer-ne còpies de seguretat amb eines normals (`tar`, `rsync`).
- Podem editar fitxers de configuració directament sense entrar al contenidor.
- Podem versionar les configuracions amb Git (els fitxers de configuració, no les dades binàries com bases de dades).

5.12 Xarxes Docker

Per defecte, tots els contenidors d'un `docker-compose.yml` es connecten a una xarxa interna que Docker crea automàticament. Aquesta xarxa permet que els contenidors es comuniquin entre ells pel **nom del servei** (no pas per IP).

Per exemple, al fitxer `docker-compose.yml` d'abans, el contenidor `uptime-kuma` podria parlar amb el `portainer` fent referència a `http://portainer:9443` des del seu interior, sense que el port 9443 estigui exposat a l'amfitrió. Això és útil per a serveis que volem que només siguin accessibles dins la xarxa Docker, no pas des de fora.

Podem definir xarxes personalitzades:

```
networks:
  frontend:
  backend:

services:
  homepage:
    networks:
      - frontend
  portainer:
    networks:
      - backend
      - frontend
```

Això ens permet segmentar l'accés: Homepage pot parlar amb Portainer (a través de la xarxa frontend), però els altres serveis no.

5.13 El fitxer .env

Les variables d'entorn sovint contenen informació sensible: contrasenyes, tokens, claus. Per això, Docker Compose pot llegir variables d'un fitxer `.env` al mateix directori:

```
TZ=Europe/Madrid
POSTGRES_PASSWORD=secret
INFLUXDB_TOKEN=elmeutoken
```

I al `docker-compose.yml` les referenciem amb `${POSTGRES_PASSWORD}`. Aquest fitxer **no s'ha de versionar amb Git** — l'afegim al `.gitignore` (Capítol 9).

5.14 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
    subgraph Host["Amfitrió (Raspberry Pi · Debian)"]
        DOCKER["Docker Engine"]
        SOCK["/var/run/docker.sock"]
        DATA["/home/bernat/homelab/data/"]
    end

    subgraph XarxaDocker["Xarxa Docker"]
        P["Portainer<br/>(contenidor)"]
        U["Uptime Kuma<br/>(contenidor)"]
        H["Homepage<br/>(contenidor)"]
    end

    subgraph Extern["Xarxa amfitrió"]
        HOSTP["Port 9443"]
        HOSTU["Port 3001"]
        HOSTH["Port 3000"]
    end

    DOCKER --> SOCK
    DOCKER --> P
    DOCKER --> U
    DOCKER --> H
    P --> DATA
    U --> DATA
    H --> DATA
    P --> HOSTP
    U --> HOSTU
    H --> HOSTH
```

5.15 Errors habituals

Error 1: executar contenidors com a root i no configurar-los bé. Síntoma: el contenidor pot fer coses perilloses perquè té privilegis de root. Solució: usar sempre `restart: unless-stopped` i, quan calgui, opcions de seguretat com `read_only`, `cap_drop`, o executar com a usuari no root.

Error 2: no persistir les dades. Síntoma: en actualitzar o esborrar un contenidor, es perden les dades. Solució: SEMPRES usar volums o bind mounts per a dades importants.

Error 3: mapejar massa ports a l'amfitrió. Síntoma: la llista de serveis exposats creix, és difícil saber què és accessible i des de on. Solució: exposar només el que cal, mantenir la comunicació interna dins la xarxa Docker.

Error 4: no mirar els logs quan alguna cosa falla. Síntoma: contenidor que no arrenca, no sabem per què. Solució: `docker compose logs servei` o `docker logs nom`. El 90% dels errors es veuen als logs.

Error 5: deixar contenidors antics consumint recursos. Síntoma: el sistema va lent, el disc s'omple. Solució: `docker container prune`, `docker image prune`, `docker volume prune`. Compte amb els volums que continguin dades.

5.16 Bones pràctiques

1. ****Usa sempre `docker compose`****, no pas `docker run` per a serveis que vagin més enllà d'una prova ràpida.
2. ****Defineix un sol `docker-compose.yml`**** per a tot el sistema, o divideix en piles (`stacks/`) per temàtica.
3. **Fes servir `bind mounts`** per a dades i configuracions a `/home/bernat/homelab/data/`.
4. ****Usa fitxers `.env`**** per a secrets, i no els versionis.
5. ****Política `restart: unless-stopped`**** per a tots els serveis que han d'estar sempre disponibles.
6. **Neteja periòdicament** amb `docker system prune`.
7. **Monitoritza** amb Uptime Kuma — que els contenidors no estiguin `running` no vol dir que estiguin funcionant correctament.

5.17 Resum

Hem après què és Docker, per què serveix, què són imatges, contenidors, volums i xarxes. Hem après les ordres essencials, hem vist per què `docker compose` és millor que `docker run` per a un homelab, i hem analitzat un `docker-compose.yml` complet del BernatLab. En el proper capítol veurem Portainer, la interfície web que ens permet gestionar tot això gràficament.

5.18 Exercicis pràctics

1. Comprova la versió de Docker: `docker --version`, `docker compose version`.
2. Llista les imatges locals: `docker images`.
3. Llista els contenidors actius: `docker ps` i `docker ps -a`.
4. Entra dins d'un contenidor: `docker exec -it uptime-kuma bash`. Mira què hi ha dins. Surt amb `exit`.
5. Mira els logs d'un servei: `docker compose logs homepage`.
6. Comprova l'ús de recursos: `docker stats` (executa durant 5 segons i prem `Ctrl+C`).
7. Crea una carpeta nova dins de `homelab/stacks/experiment/`, escriu-hi un `docker-compose.yml` amb un servidor Nginx i aixeca'l.
8. Fes `docker compose down` i comprova que el contenidor ha desaparegut.
9. Comprova l'espai ocupat per Docker: `docker system df`.

Comandes útils:

```
docker ps, docker ps -a
docker images, docker pull, docker rmi
```

```
docker logs, docker logs -f
docker exec -it nom bash
docker stats
docker volume ls, docker network ls
docker compose up -d, docker compose down
docker compose logs, docker compose pull
```

Paraules clau: **imatge, contenidor, volum, xarxa, port, bind mount, env, restart, docker compose, prune, persistència, portainer, uptime-kuma, homepage.**

Capítol 6 — Portainer

*“Portainer és la finestra al cor de Docker. Sense ella, hem de saber totes les ordres; amb ella, podem tocar-ho tot amb el ratolí.”**

6.1 Què és Portainer

Portainer Community Edition (CE) és una **interfície web de codi obert** per gestionar entorns Docker. Ens permet, des d'un navegador, veure i controlar contenidors, imatges, volums, xarxes, piles, serveis i configuracions — tot allò que, des de la línia d'ordres, requeriria desenes de comandes diferents.

Portainer va néixer el 2016 com una eina per a administradors que volien una manera més visual de gestionar contenidors. Avui és l'eina estàndard de facto per a homelabs i petites empreses. La versió CE (gratuïta) cobreix tot el que necessitem al BernatLab. La versió comercial (Business) afegeix funcionalitats que, per al nostre cas, serien supèrflues.

Portainer es connecta al **socket de Docker** (`/var/run/docker.sock`), que és l'API interna que Docker exposa per a la gestió. A través d'aquest socket, Portainer pot fer exactament el que faríem nosaltres des de la consola, però amb una interfície gràfica, gràfics, formularis, cerques, filtres.

6.2 Per què l'utilitzem

Hi ha servidors professionals que opten per no instal·lar Portainer i gestionar-ho tot des de la consola. Té els seus arguments: una eina menys, una superfície d'atac menor, un coneixement més profund de Docker. Però al BernatLab, Portainer ens aporta molt:

- **Visibilitat immediata.** Obrim el navegador i veiem tots els contenidors, el seu estat, l'ús de CPU/RAM, els ports exposats, les dates de creació. En un cop d'ull, sabem com està el sistema.
- **Operacions ràpides.** Reiniciar un contenidor, veure els seus logs, inspeccionar el seu sistema de fitxers, accedir a una consola — tot amb un parell de clics.
- **Punt d'entrada per a gent no tècnica.** Si algun dia algú altre ha de tocar el sistema (un company, un familiar), Portainer és molt més amable que la consola.
- **Documentació visual.** Quan expliquem com és el BernatLab, una captura de Portainer val més que mil descripcions.

Això no vol dir que abandonem la consola. Al contrari: Portainer ens servirà per a operacions puntuals, consulta ràpida, i per ensenyar. Per a canvis importants (configuracions, actualitzacions, desplegaments) continuarem treballant amb fitxers `docker-compose.yml` i ordres de línia de comandes.

6.3 Instal·lació al BernatLab

Portainer ja està instal·lat a `https://100.115.134.76:9443`. Vegem com s'ha fet i què hi ha configurat.

Definició al docker-compose.yml

```
services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: unless-stopped
    ports:
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /home/bernat/homelab/data/portainer:/data
```

Tres aspectes a notar:

1. **El port 9443** és el port HTTPS de Portainer. Noteu que no és el 9000 estàndard, sinó el 9443, que és el port segur. El 9000 encara existeix com a alternativa HTTP, però el 9443 és el recomanat.
2. ****/var/run/docker.sock****: muntat dins del contenidor. Això és el que permet a Portainer parlar amb el dimoni de Docker de l'amfitrió. Sense això, Portainer no podria fer res.
3. ****/home/bernat/homelab/data/portainer****: bind mount a la nostra carpeta, on es guardarà la configuració i la base de dades de Portainer.

Primer accés

La primera vegada que entrem a <https://100.115.134.76:9443>, el navegador ens mostrarà un avís de certificat no vàlid — és normal, perquè Portainer genera un certificat autosignat en instal·lar-se. Cal acceptar l'avís (a Chrome, "Configuració avançada → Continua").

A continuació, ens demanarà crear un usuari administrador. Triarem una contrasenya forta, perquè Portainer té el control absolut del Docker de la màquina.

Un cop dins, ens portarà a la **vista local** (Local environment), que és l'entorn Docker de la nostra Raspberry.

6.4 Interfície: vista general

Quan entrem, veiem el **dashboard** amb:

- Una graella de targetes amb estadístiques: nombre de contenidors actius, aturats, imatges, volums, xarxes.
- Una gràfica d'ús de recursos (CPU, memòria).
- Un llistat dels contenidors en execució.

A la part superior hi ha la **barra de navegació** amb:

- **Home**: vista general.
- **Stacks**: agrupacions de serveis basades en fitxers Compose.
- **Containers**: tots els contenidors, actius i aturats.
- **Images**: imatges Docker presents a la màquina.
- **Volumes**: volums i bind mounts.
- **Networks**: xarxes Docker.

- **Configs:** configuracions de Docker Swarm (no les farem servir).
- **Events:** registre d'esdeveniments del sistema.

A l'esquerra, una barra lateral ens permet canviar d'entorn (per defecte, l'entorn local) i accedir a la configuració.

6.5 Gestió de contenidors

La secció **Containers** és la que més farem servir. Aquí veiem tots els contenidors, amb columnes que ens indiquen:

- **Nom:** l'identificador del contenidor.
- **Estat:** running, stopped, paused, exited.
- **Imatge:** la imatge a partir de la qual es va crear.
- **Ports:** ports exposats a l'amfitrió.
- **Acció:** botons ràpids.

Si cliquem sobre un contenidor concret, accedim a una vista detallada amb:

- **Logs:** sortida estàndard i d'error del contenidor, en directe. Molt útil per diagnosticar errors.
- **Inspect:** la configuració completa del contenidor en format JSON.
- **Stats:** ús de CPU, memòria, xarxa, en temps real.
- **Console:** una consola dins del contenidor (`/bin/sh` o `/bin/bash`, segons la imatge).
- **Volumes:** la llista de volums i bind mounts del contenidor.
- **Network:** les xarxes a les quals està connectat.

Accions habituals

- **Start / Stop / Restart:** arrencar, aturar, reiniciar un contenidor.
- **Kill:** aturar de forma forçada (equivalent a `docker kill`).
- **Pause / Unpause:** pausar temporalment.
- **Remove:** esborrar el contenidor.
- **Recreate:** tornar a crear el contenidor amb la mateixa configuració.
- **Duplicate / Export:** opcions avançades que rarament usarem.

A la pràctica, el 80% del temps estarem mirant **Logs** (per entendre errors) i fent **Restart** (per recuperar serveis).

6.6 Stacks: una capa d'organització

Una **Stack** a Portainer és un grup de serveis definits en un fitxer `docker-compose.yml`. La gràcia és que podem:

- Crear una nova stack des del navegador, enganxant un fitxer `docker-compose.yml`.

- Desplegar-la amb un clic.
- Editar-la visualment.
- Aturar-la, reiniciar-la, esborrar-la.

Al BernatLab, podem organitzar les piles per temàtica:

```
homelab/
└── stacks/
    ├── core/      → portainer, homepage, uptime-kuma
    ├── monitoring/ → grafana, influxdb
    ├── data/      → postgres, filebrowser
    ├── iot/       → mosquitto, node-red
    └── media/     → pi-hole, nginx-proxy
```

Cadascuna amb el seu `docker-compose.yml`. A Portainer, les podem veure totes, cadascuna amb el seu estat.

6.7 Imatges

La secció **Images** ens mostra totes les imatges Docker presents a la màquina. Podem:

- Veure mida, data de creació, etiqueta, ID.
- **Pull** una nova imatge des d'un registre.
- **Build** una imatge a partir d'un Dockerfile.
- **Push** una imatge a un registre.
- **Remove** imatges que no usem.
- **Prune** per netejar les imatges penjant (dangling).

Al BernatLab, és bona pràctica fer un **prune** periòdicament per alliberar espai. Les imatges no usades es poden acumular ràpidament, especialment després d'actualitzacions.

6.8 Volumes

La secció **Volumes** ens permet gestionar l'emmagatzematge persistent. Hi trobarem:

- **Volumes anomenats** (els que Docker crea automàticament quan usem `volumes: nom_volum`).
- **Bind mounts** apareixen aquí, encara que tècnicament no són volums gestionats per Docker, sinó carpetes de l'amfitrió.

Podem veure la mida de cada volum (útil per entendre què ocupa espai), inspeccionar-los, i esborrar-los. Compte: esborrar un volum equival a perdre les dades que conté. Feu-ho només si esteu segurs.

6.9 Xarxes

La secció **Networks** ens mostra les xarxes Docker. Per defecte, n'hi ha tres:

- **bridge**: la xarxa per defecte, a la qual es connecten tots els contenidors que no especifiquem una xarxa.
- **host**: una xarxa especial que fa que el contenidor comparteixi la xarxa de l'amfitrió (no l'usarem).

- **none**: sense xarxa.

Quan usem `docker compose`, es crea una xarxa nova per a cada projecte. A Portainer les podem veure totes.

6.10 Configuració i seguretat

A la configuració de Portainer podem:

- Canviar la contrasenya de l'usuari administrador.
- Afegir altres usuaris (útil si volem donar accés a algú amb permisos limitats).
- Definir equips (teams) per organitzar accessos.
- Configurar el registre d'esdeveniments.
- Veure les estadístiques d'ús de Portainer.

Recomanació de seguretat: si no l'estem fent servir, podem aturar temporalment el contenidor. Però compte: si l'aturem i volem tornar a entrar, ho haurem de fer des de la consola amb `docker compose up -d`. Una bona política és mantenir Portainer sempre actiu, però limitar-ne l'accés a la xarxa Tailscale — cosa que ja fem, perquè el port 9443 no està exposat a Internet.

6.11 Quan NO usar Portainer

Portainer és una eina fantàstica, però hi ha casos en què la consola és millor:

- **Edició de fitxers de configuració.** Canviar el `docker-compose.yml` directament al fitxer de l'amfitrió és més transparent que fer-ho des de la interfície de Portainer, que desa el fitxer a la seva pròpia base de dades.
- **Automatització.** Si volem scriptar accions, hem d'usar la consola o l'API de Docker, no pas la interfície gràfica.
- **Aprenentatge.** Si volem entendre bé Docker, hem de fer servir les seves ordres, no pas una eina que les amaga.

Al BernatLab, **la consola és la nostra eina principal** i Portainer és el complement visual. Aquest és l'equilibri correcte.

6.12 Exemples d'operacions

Veure els logs en directe d'un contenidor

A Portainer: Containers → uptime-kuma → Logs → Live (botó de seguiment).

Equivalent a la consola:

```
docker logs -f uptime-kuma
```

Inspeccionar un contenidor

A Portainer: Containers → portainer → Inspect.

Equivalent a la consola:

```
docker inspect portainer
```


Obrir una consola dins d'un contenidor

A Portainer: Containers → homepage → Console → connectar.

Equivalent a la consola:

```
docker exec -it homepage bash
```

Veure l'ús de recursos en temps real

A Portainer: Containers → un contenidor qualsevol → Stats.

Equivalent a la consola:

```
docker stats
```

Netejar imatges antigues

A Portainer: Images → Prune.

Equivalent a la consola:

```
docker image prune -a
```

6.13 Errors habituals

Error 1: oblidar que Portainer pot fer molt de mal. Si accidentalment esborrem un volum, podem perdre dades. Compte amb l'opció **Remove** — assegureu-vos de què esteu esborrant.

Error 2: deixar Portainer exposat a Internet. Si per error exposem el port 9443 a la xarxa pública, algú podria accedir-hi i fer malbé tot el sistema. Al BernatLab, el port 9443 només és accessible des de Tailscale.

Error 3: confiar massa en Portainer i oblidar la consola. Si Portainer falla (per exemple, per un error en una actualització), necessitem poder arreglar les coses des de la consola. Per això, sempre hem de saber què està passant per sota.

6.14 Resum

Portainer ens dona una interfície visual per gestionar tot el sistema Docker de la Raspberry: contenidors, imatges, volums, xarxes, piles. L'hem d'usar com a complement de la consola, no pas com a substitut. Al BernatLab ja el tenim configurat i és la nostra eina de consulta ràpida. En el proper capítol veurem Uptime Kuma, l'eina que ens avisa quan alguna cosa falla.

6.15 Exercicis pràctics

1. Entra a <https://100.115.134.76:9443> i explora el dashboard.
2. Compta quants contenidors hi ha actius. Compara'l amb la sortida de `docker ps`.
3. Mira els logs en directe d'Homepage durant 30 segons.
4. Entra dins del contenidor de Uptime Kuma amb la consola de Portainer i executa `ls /app`. Què hi ha?
5. A la secció Imatges, comprova quantes imatges hi ha i quant ocupen en total.

6. A la secció Volumes, identifica els volums de cada servei. Quins són bind mounts i quins són volums gestionats per Docker?

7. Fes un **prune** d'imatges no usades. Compte! Primer comprova que tens espai.

Comandes útils equivalents a accions de Portainer:

```
docker ps                # Containers
docker logs -f nom       # Logs en directe
docker inspect nom       # Detalls
docker exec -it nom bash  # Consola
docker stats             # Recursos
docker image prune -a     # Neteja imatges
```

Paraules clau: **Portainer, dashboard, contenidor, logs, exec, prune, socket, stack, bind mount, interfície gràfica, homelab.**

Capítol 7 — Uptime Kuma

""Un servei que no saps si està caigut és un servei que ja està caigut. Però no t'has adonat.""

7.1 Què és Uptime Kuma

Uptime Kuma és una eina de **monitorització autoallotjada** de codi obert. La seva feina és, essencialment, fer comprovacions periòdiques sobre serveis que nosaltres definim i avisar-nos quan alguna cosa falla. Està escrita en Node.js, és lleugera, té una interfície neta, i és una de les millors opcions gratuïtes per a homelabs.

La paraula **uptime** en anglès vol dir "temps de funcionament". Un servei amb un uptime del 99,9% ha estat disponible el 99,9% del temps en un període determinat. Uptime Kuma ens ajuda a mesurar aquest número — i a millorar-lo, perquè ens avisa quan baixa.

Uptime Kuma va ser creada per Louis Cheung, un desenvolupador de Hong Kong, i publicada el 2021. Des de llavors, ha esdevingut l'estàndard de facto per a la monitorització autoallotjada, amb una comunitat activa i una quantitat enorme de funcionalitats.

7.2 Per què ens interessa

Al BernatLab tenim diversos serveis que volem que estiguin sempre disponibles: Portainer per gestionar, Uptime Kuma mateix (meta-monitorització), Homepage com a porta d'entrada, i — més endavant — la base de dades, Node-RED, l'API de sensors, etc. Sense monitorització, no sabríem si algun d'aquests serveis ha caigut fins que intentéssim accedir-hi i no poguéssim. I sovint, quan un usuari s'adona, ja fa estona que el servei no funciona.

Uptime Kuma ens permet:

- Saber en **temps real** l'estat de cada servei.
- Rebre **alertes** quan un servei cau o es recupera.
- Mesurar el **temps de resposta** dels serveis.
- Guardar un **historial** d'incidències.
- Generar una **pàgina pública d'estat** per a qui vulgui consultar-la.
- Rebre notificacions per **Telegram, correu, Discord, Slack, webhook**, etc.

7.3 Instal·lació al BernatLab

Uptime Kuma ja està instal·lat a `http://100.115.134.76:3001`. Vegem com està configurat.

Definició al docker-compose.yml

```
services:
  uptime-kuma:
    image: louislam/uptime-kuma:latest
    container_name: uptime-kuma
    restart: unless-stopped
    ports:
      - "3001:3001"
    volumes:
      - /home/bernat/homelab/data/uptime-kuma:/app/data
```

Aspectes a notar:

- La imatge `louislam/uptime-kuma:latest` és l'oficial.
- Muntem un bind mount a `/home/bernat/homelab/data/uptime-kuma` per persistir la configuració i l'històric.
- No exposem cap port intern — el 3001 és el port de Uptime Kuma i l'exposem a l'amfitrió.

Primer accés

Quan entrem per primera vegada a `http://100.115.134.76:3001`, Uptime Kuma ens demana crear un compte d'administrador. Triarem una contrasenya forta. A partir d'aquí, totes les dades (usuaris, monitors, configuracions) es guarden a la base de dades interna del contenidor, que està al bind mount que hem definit.

7.4 Tipus de monitors

Uptime Kuma suporta una quantitat impressionant de tipus de monitor. Els que farem servir al BernatLab són:

HTTP(s)

El tipus més comú. Uptime Kuma fa una petició HTTP/HTTPS a una URL i ens diu:

- Si la petició ha retingut resposta.
- El codi d'estat (200, 404, 500, etc.).
- El temps de resposta.
- El cos de la resposta (opcional).

Serveix per monitoritzar pàgines web, APIs, serveis que escolten HTTP. Per defecte, busca un codi 2xx com a èxit. Podem canviar-ho per acceptar només 200, o per acceptar qualsevol cosa menys 5xx.

Ping

Uptime Kuma envia un ping ICMP a una adreça IP o nom de domini. Si rep resposta, el servei està viu. No ens diu res sobre el servei concret, només sobre la connectivitat. Serveix per monitoritzar màquines, dispositius de xarxa, o simplement comprovar que la Raspberry està en marxa.

Port TCP

Uptime Kuma intenta obrir una connexió TCP a un port concret. Si la connexió s'obre amb èxit, el servei està escoltant. Serveix per monitoritzar serveis que no parlen HTTP (per exemple, una base de dades al port 5432, MQTT al 1883).

DNS

Comprova que un nom de domini es resol correctament a una IP. Serveix per monitoritzar el DNS mateix o per alertar si un domini canvia d'IP.

Certificat SSL/TLS

Comprova la validesa i expiració d'un certificat HTTPS. Ens pot avisar X dies abans que caduqui, la qual cosa és molt útil per a serveis amb certificats propis (no pas per a serveis darrere de Tailscale, que no exposen HTTPS directament).

Paraula clau (Keyword)

Una variació del monitor HTTP: Uptime Kuma cerca una paraula concreta a la resposta. Si la troba, el servei està funcionant correctament; si no, alguna cosa va malament. Serveix per fer comprovacions més fines que el simple codi d'estat.

Altres

Hi ha més tipus: **Docker container** (comprova si un contenidor concret està en marxa), **Steam Game Server**, **MQTT**, **SQL Server**, **Push** (per a monitors que es disparen des d'una font externa), **gRPC**, etc. Al BernatLab, els bàsics (HTTP i Ping) ens cobriran la majoria dels casos.

7.5 Configurar monitors al BernatLab

Vegem quins monitors hauríem de tenir configurats al BernatLab i quins valors posar.

1. La pròpia Raspberry (Ping)

- **Tipus:** Ping
- **Nom:** "Raspberry Pi (hortosona)"
- **Host:** `hortosona` (MagicDNS de Tailscale) o `100.115.134.76`
- **Interval:** 60 segons
- **Timeout:** 5 segons
- **Retries:** 3 (abans d'alertar)

Aquest monitor ens avisa si la Raspberry deixa de respondre a pings. Compte: si la Raspberry penja totalment, el monitor quedarà caigut però no podrem rebre l'alerta perquè el correu/Telegram depenen d'ella. En aquest cas, podem complementar amb un **monitor extern** (Uptime Kuma al núvol, o un servei com UptimeRobot) que ens avisi si la Tailscale IP deixa de respondre des de fora.

2. Portainer (HTTP)

- **Tipus:** HTTP(s)
- **Nom:** "Portainer"

- **URL:** `https://100.115.134.76:9443`
- **Interval:** 60 segons
- **Acceptar codis 2xx** (per defecte)

Això fa una petició HTTPS a Portainer. Compte: Uptime Kuma potser no validarà el certificat autosignat. Podem desactivar la validació SSL al monitor.

3. Uptime Kuma (HTTP)

- **Tipus:** HTTP(s)
- **Nom:** "Uptime Kuma (self)"
- **URL:** `http://100.115.134.76:3001`
- **Interval:** 60 segons

Meta-monitorització. Útil per confirmar que el sistema de monitorització funciona.

4. Homepage (HTTP)

- **Tipus:** HTTP(s)
- **Nom:** "Homepage"
- **URL:** `http://100.115.134.76:3000`
- **Interval:** 60 segons

5. Hort Osona (HTTP, web pública)

- **Tipus:** HTTP(s)
- **Nom:** "Hort Osona"
- **URL:** `https://bernadmora.github.io/hort-osona/`
- **Interval:** 300 segons (5 minuts, per no carregar GitHub Pages)
- **Acceptar codis 2xx**

Aquest monitor ens avisa si la web pública deixa de funcionar.

6. Tailscale (Ping extern)

- **Tipus:** Ping
- **Nom:** "Tailscale connectivity"
- **Host:** `100.115.134.76` o un nom de Tailscale
- **Interval:** 300 segons

Comprova que la xarxa Tailscale continua activa.

7.6 Configurar alertes

Aquí és on Uptime Kuma demostra la seva potència. Podem configurar notificacions per múltiples canals. Vegem com configurar **Telegram**, que és el canal que farem servir al BernatLab.

Configurar un bot de Telegram

1. Parlem amb **@BotFather** a Telegram.
2. Li diem `/newbot` i seguim les instruccions: posem un nom i un nom d'usuari (que ha d'acabar en bot).
3. Rebem un **token** llarg, del tipus `123456789:ABCDefghIJKlmnoPQRsTUVwxyz`. **Guardem aquest token en lloc segur.**
4. Iniciem una conversa amb el nostre bot (li enviem qualsevol missatge).
5. Per obtenir el nostre **chat ID**, podem: - Usar `@userinfobot`, que ens el donarà. - O fer una petició a `https://api.telegram.org/bot<TOKEN>/getUpdates` i buscar l'ID al JSON retornat.

Configurar la notificació a Uptime Kuma

A Uptime Kuma:

1. Anem a **Settings** → **Notifications**.
2. Cliquem **Setup Notification**.
3. Triem **Telegram**.
4. Omplim: - **Bot Token**: el token que hem rebut. - **Chat ID**: el nostre chat ID.
5. Cliquem **Test** per comprovar que funciona. Hauríem de rebre un missatge del bot.
6. Desem.

A partir d'ara, podem seleccionar aquesta notificació per a qualsevol monitor. Quan el monitor passi de "up" a "down", Uptime Kuma enviarà un missatge a Telegram. Quan torni a "up", rebré un altre missatge informant de la recuperació.

Bones pràctiques amb alertes

- **No configurar alertes per a tot.** Si rebem 50 missatges al dia, deixarem de mirar-los. Configurem alertes només per als serveis crítics.
- **Definir el llindar de retries correcte.** Si un monitor fa 3 comprovacions i falla, podem configurar que ho faci 1 vegada abans d'alertar (per evitar falss positius).
- **Configurar finestres de manteniment.** Si sabem que anem a fer una aturada, podem silenciar les alertes temporalment.
- **Combinar canals.** Telegram per a alertes immediates, correu per a informes setmanals.

7.7 Pàgina d'estat pública

Uptime Kuma ens permet generar una **pàgina d'estat pública** on qualsevol pot veure l'estat dels nostres serveis, sense necessitat d'autenticació. És útil si volem:

- Compartir l'estat del BernatLab amb altres persones.
- Tenir una pàgina pública del tipus `status.bernatlab.cat` (en el futur, quan tinguem un domini).

Per activar-la, anem a **Settings** → **Status Page**, creem una pàgina nova, afegim els monitors que volem que siguin visibles, i obtenim una URL pública.

A la pràctica, al BernatLab, podem crear una pàgina d'estat interna per al nostre ús, sense fer-la pública a Internet.

7.8 Manteniment i actualitzacions

Uptime Kuma evoluciona activament. Per mantenir-lo actualitzat:

```
cd /home/bernat/homelab
docker compose pull uptime-kuma
docker compose up -d uptime-kuma
```

Les actualitzacions solen ser suaus i no trenquen la configuració. De tota manera, és bona pràctica fer una còpia de seguretat abans:

```
tar czf ~/homelab/backup/uptime-kuma-$(date +%F).tar.gz \
~/homelab/data/uptime-kuma
```

7.9 Comprendre les estadístiques

Uptime Kuma ens dóna molta informació útil:

- **Uptime %**: percentatge de temps que el servei ha estat disponible. Es calcula sobre una finestra mòbil (per defecte, els últims 90 dies).
- **Avg. response time**: temps de resposta mitjà.
- **Cert exp**: data d'expiració del certificat SSL.
- **Historial**: línia de temps amb pujades, baixades, incidents.

Aquesta informació és valuosa per:

- Detectar serveis que es degraden lentament.
- Planificar capacitats (si els temps de resposta pugen, potser el sistema va saturat).
- Demostrar estabilitat (per exemple, davant d'un proveïdor de núvol).

7.10 Exemple d'incident: què fer

Imagineu que Uptime Kuma ens avisa per Telegram: "Portainer is down". Què fem?

1. **Comprovar**: el missatge és correcte? És un fals positiu? Esperem un parell de minuts a veure si es recupera sol.
2. **Mirar logs**: `docker logs portainer` o `docker compose logs portainer`. Sovint el problema és obvi.
3. **Reiniciar**: `docker compose restart portainer`. Soluciona el 80% dels casos.
4. **Si no es recupera**: accedir a Portainer via... oh, espera, Portainer és el que falla. Cal entrar per SSH directament.
5. **Escalar**: mirar Uptime Kuma, mirar la càrrega del sistema amb `htop`, mirar l'espai de disc amb `df -h`.

6. **Documentar:** un cop resultat, escriure al CHANGELOG què ha passat i per què.

Aquesta és la rutina de resposta a incidents. No cal ser expert, només cal seguir els passos amb calma.

7.11 Esquema de monitorització

Esquema Mermaid (text):

```
graph LR
    subgraph BernatLab["BernatLab"]
        KUMA["Uptime Kuma<br/>(3001)"]
    end

    subgraph Objectius["Objectius a monitoritzar"]
        RPI["Raspberry Pi<br/>(ping)"]
        PORT["Portainer<br/>(HTTPS:9443)"]
        HOME["Homepage<br/>(HTTP:3000)"]
        HORT["Hort Osona<br/>(HTTPS)"]
        TAIL["Tailscale<br/>(ping)"]
    end

    subgraph Alertes["Canals d'alerta"]
        TG["Telegram bot"]
        MAIL["Correu electrònic"]
    end

    KUMA -->|ping| RPI
    KUMA -->|https| PORT
    KUMA -->|http| HOME
    KUMA -->|https| HORT
    KUMA -->|ping| TAIL

    RPI -->|caigut| KUMA
    PORT -->|caigut| KUMA
    HOME -->|caigut| KUMA
    HORT -->|caigut| KUMA
    TAIL -->|caigut| KUMA

    KUMA -->|alerta| TG
    KUMA -->|alerta| MAIL
```

7.12 Errors habituals

Error 1: configurar alertes per a tot. Síntoma: rebem tants missatges que els ignorem. Solució: només alertes per a serveis crítics, i combinar amb finestres silenciades.

Error 2: no configurar retries. Síntoma: rebem alertes falses per pèrdues puntuals de xarxa. Solució: configurar `retries`: 3 o més, perquè el servei hagi de fallar diverses vegades consecutives abans d'alertar.

Error 3: monitoritzar serveis que depenen d'altres serveis. Síntoma: quan un servei cau, molts monitors entren en alerta alhora. Solució: estructurar bé els monitors, acceptar que algunes dependències són inevitables.

Error 4: no guardar l'històric. Síntoma: volem veure què va passar fa dues setmanes i Uptime Kuma ja ho ha esborrat. Solució: augmentar la finestra d'històric, o exportar periòdicament.

7.13 Resum

Uptime Kuma és el cor de la monitorització del BernatLab. Ens permet veure en temps real l'estat dels nostres serveis, rebre alertes quan alguna cosa falla, i mesurar el temps de resposta. Combinat amb Telegram, ens dóna un sistema d'alertes eficaç i lleuger. En el proper capítol veurem Homepage, el panell d'entrada al BernatLab.

7.14 Exercicis pràctics

1. Entra a `http://100.115.134.76:3001` i revisa la configuració dels monitors.
2. Comprova quin és l'uptime de cada monitor.
3. Crea un nou monitor de tipus Ping per a una adreça IP externa, com `1.1.1.1`.
4. Configura una alerta per Telegram (si no la tens configurada).
5. Fes una prova: atura un dels contenidors amb `docker stop portainer`, espera un parell de minuts, comprova que Uptime Kuma ha detectat la caiguda, rep l'alerta a Telegram, i reinicia el contenidor.
6. Mira l'historial d'un monitor i compta quantes vegades ha estat caigut en els últims 30 dies.

Comandes útils:

```
docker logs uptime-kuma
docker compose restart uptime-kuma
docker compose pull uptime-kuma
```

Paraules clau: **monitorització, uptime, ping, HTTP, HTTPS, certificat SSL, Telegram, alerta, retries, status page, fallada, temps de resposta.**

Capítol 8 — Homepage

"Quan algú entra al BernatLab, el primer que ha de veure és ordre. Homepage és el porter que dóna la benvinguda."

8.1 Què és Homepage

Homepage és una aplicació web moderna, escrita en Next.js, que ens permet crear un **panell d'entrada** personalitzat per al nostre servidor. La pàgina mostra una graella de targetes, cadascuna de les quals és un enllaç directe a un dels nostres serveis. A més, pot incloure **widgets** que mostren informació en temps real: ús de CPU, memòria, temperatura, estat dels contenidors, estadístiques diverses.

Homepage va ser creada per iurylab, un desenvolupador europeu, i publicada el 2022. Des de llavors, s'ha convertit en una de les aplicacions d'autoallotjament més populars, amb milers d'estrelles a GitHub, una comunitat activa, i una documentació excel·lent.

La gràcia de Homepage és la seva **configuració totalment basada en fitxers YAML**. No hi ha base de dades externa, no hi ha interfície d'administració web: tot es configura editant fitxers, que viuen dins del contenidor (o al bind mount que hem definit). Això pot semblar poc amigable al principi, però en realitat és una virtut: podem versionar la configuració amb Git, podem replicar-la en una altra màquina, podem entendre exactament què està passant.

8.2 Per què l'utilitzem al BernatLab

Al BernatLab tenim, avui, tres serveis principals: Portainer, Uptime Kuma i Homepage mateix. Més endavant, tindrem Grafana, InfluxDB, Node-RED, File Browser, una base de dades, l'API de sensors, etc. Si entrem al servidor per la IP Tailscale, què trobem?

Sense Homepage, hauríem de recordar totes les adreces i ports:

<https://100.115.134.76:9443> per a Portainer, <http://100.115.134.76:3001> per a Uptime Kuma, etc. Un malson.

Amb Homepage, posem <http://100.115.134.76:3000> i veiem una pàgina maca amb totes les targetes organitzades. Un clic, i entrem al servei. A més, podem veure d'un cop d'ull quin servei està caigut, quina temperatura té la CPU, quanta RAM queda.

Homepage és, per tant, la **porta d'entrada visual** al BernatLab. També és un bon lloc per mostrar el sistema a visites (o a un mateix, per fer-se'n una foto mental).

8.3 Instal·lació al BernatLab

Homepage ja està instal·lat a <http://100.115.134.76:3000>. Vegem com està configurat.

Definició al docker-compose.yml

```
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - HOMEPAGE_ALLOWED_HOSTS=gethomepage.local:3000
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - /home/bernat/homelab/data/homepage:/app/config
      - /home/bernat/homelab/stacks/homepage:/app/config/widgets
```

Detalls importants:

- **Imatge:** `ghcr.io/gethomepage/homepage:latest`. Compte: la imatge oficial NO és a Docker Hub, sinó a GitHub Container Registry. Si busquem a Docker Hub, trobarem imatges no oficials que poden estar desactualitzades.
- **Port 3000:** el port estàndard de Homepage.
- **Variable `HOMEPAGE_ALLOWED_HOSTS`:** explicada més avall.
- **Tres volums:**
 - `/var/run/docker.sock`: per accedir a la informació de Docker (contenidors, volums, etc.).
 - `/home/bernat/homelab/data/homepage`: on viuen els fitxers de configuració principals.
 - `/home/bernat/homelab/stacks/homepage`: on posarem els fitxers dels widgets personalitzats.

8.4 El socket de Docker: per què és tan important

Un dels aspectes clau de Homepage és que pot mostrar informació sobre els nostres **contenidors en temps real**: si estan corrent, quina CPU consumeixen, etc. Per fer-ho, necessita accedir al socket de Docker de l'amfitrió.

El socket és un fitxer especial a `/var/run/docker.sock` que Docker exposa com a API. Quan muntem aquest fitxer dins del contenidor de Homepage (amb la línia `/var/run/docker.sock:/var/run/docker.sock:ro`), estem donant a Homepage permisos per **llegir** l'estat de Docker. La **ro** (read-only) és important: així Homepage només pot veure, no pas canviar coses.

Això és molt poderós, però també és un risc potencial. Si algú entrés a Homepage amb intencions malicioses, podria obtenir informació detallada sobre el sistema. Per això:

- Muntem el socket en mode només lectura.
- Mantenim Homepage en una xarxa privada (Tailscale).
- Estem atents a actualitzacions de seguretat.

8.5 HOMEPAGE_ALLOWED_HOSTS: el host validation

Aquesta és una de les configuracions més importants i menys enteses de Homepage. La variable `HOMEPAGE_ALLOWED_HOSTS` indica a quins **noms de host** o adreces pot servir la interfície Homepage.

Per què existeix? Per defecte, Next.js (el framework sobre el qual corre Homepage) bloqueja les peticions que arriben amb un `Host` header que no coincideix amb cap dels seus hosts coneguts. Això és una **mesura de seguretat** per evitar atacs de tipus **host header injection**.

Al BernatLab, hem configurat:

```
HOMEPAGE_ALLOWED_HOSTS=gethomepage.local:3000
```

Això vol dir que si entrem a Homepage amb l'URL `http://gethomepage.local:3000`, funcionarà. Si entrem amb una altra URL, com `http://100.115.134.76:3000`, rebré un error 400 de Next.js: "Invalid host header".

Hi ha dues solucions possibles:

1. ****Configurar DNS perquè `gethomepage.local` apunti a la IP de la Raspberry****. Això es pot fer afegint una entrada al DNS local, o simplement al fitxer `hosts` de la nostra màquina client.
2. **Modificar `HOMEPAGE_ALLOWED_HOSTS` per permetre múltiples hosts**. Per exemple: ``
`HOMEPAGE_ALLOWED_HOSTS=gethomepage.local:3000,100.115.134.76:3000,hortosona:3000`
 ``

Al BernatLab, segon la documentació oficial, la manera correcta és:

```
HOMEPAGE_ALLOWED_HOSTS=gethomepage.local:3000,100.115.134.76:3000
```

Així podem accedir tant per la IP Tailscale com pel nom `gethomepage.local` (si tenim el DNS configurat).

També podem posar `*` per permetre tots els hosts, però això és menys segur:

```
HOMEPAGE_ALLOWED_HOSTS=*
```

A la pràctica, al BernatLab, si veiem l'error "Invalid host header", sabem que hem de revisar aquesta variable.

8.6 L'estructura de configuració

La configuració de Homepage viu a `/home/bernat/homelab/data/homepage` (el bind mount). Dins hi trobem:

```
/home/bernat/homelab/data/homepage/
■■■ settings.yaml          # configuració general (tema, fons, etc.)
■■■ bookmarks.yaml        # enllaços externs
■■■ services.yaml         # definició dels serveis
■■■ widgets/              # configuració dels widgets
■   ■■■ resources.yaml    # CPU, RAM, etc.
■   ■■■ docker.yaml       # estat dels contenidors
■   ■■■ uptime.yaml       # integració amb Uptime Kuma
■   ■■■ ...
■■■ ...
```

Quan editem un fitxer, els canvis s'apliquen automàticament — Homepage llegeix els fitxers cada pocs segons i actualitza la interfície.

settings.yaml: configuració general

```
title: BernatLab
background:
  opacity: 50
  blur: sm
  saturate: 100
  brightness: 50
  image: https://images.unsplash.com/...
theme: dark
color: slate
headerStyle: boxed
```

Aquest fitxer controla el títol de la pàgina, el fons (podem posar una imatge, un color sòlid, o un degradat), el tema (clar/fosc), el color d'accent, l'estil de la capçalera.

services.yaml: la llista de serveis

```
---
- Gestió:
  - Portainer:
    href: https://100.115.134.76:9443
    description: Gestió Docker
    icon: portainer
    siteMonitor: http://100.115.134.76:9443

  - Homepage:
    href: http://100.115.134.76:3000
    description: Aquesta pàgina
    icon: homepage
    siteMonitor: http://100.115.134.76:3000

- Monitorització:
  - Uptime Kuma:
    href: http://100.115.134.76:3001
    description: Monitoratge de serveis
    icon: uptimekuma
    siteMonitor: http://100.115.134.76:3001

  - Hort Osona:
    href: https://bernatmora.github.io/hort-osona/
    description: Projecte hort familiar
    icon: hortosona
    siteMonitor: https://bernatmora.github.io/hort-osona/
```

La sintaxi és simple:

- Cada entrada és un **grup** (per exemple, "Gestió", "Monitorització").
- Dins de cada grup, hi ha **serveis**, cadascun amb:
 - **Nom**: el que es mostra a la targeta.
 - **href**: l'enllaç.
 - **description**: una descripció breu.
 - **icon**: la icona. Pot ser el nom d'un icon pack predefinit (com `portainer`, `uptimekuma`) o una URL externa.
 - **siteMonitor**: si volem que la targeta mostri l'estat del servei (integració amb Uptime Kuma o ping).

8.7 Widgets: informació en temps real

A més de les targetes, Homepage pot mostrar **widgets** que aporten informació dinàmica. Hi ha molts widgets disponibles:

Widget de recursos del sistema

Mostra CPU, memòria, disc, xarxa del sistema amfitrió. Es connecta al socket de Docker per obtenir les dades.

```
# widgets/resources.yaml
resources:
  cpu: true
  memory: true
  disk: /
  network: true
  uptime: true
  temperature: true
  load: 5
```

Això ens mostra una secció amb l'ús actual de recursos, molt útil d'un cop d'ull.

Widget de Docker

Mostra l'estat dels contenidors: quins estan corrent, quins aturats, quins han caigut.

```
# widgets/docker.yaml
docker:
  server:
    socket: /var/run/docker.sock
  containers:
    onlyAvailable: true
```

Widget d'Uptime Kuma

Integra l'estat dels monitors d'Uptime Kuma directament a Homepage. Per configurar-lo:

1. A Uptime Kuma, creem una **Status Page** pública.
2. A Homepage, configurem el widget amb l'URL d'aquesta pàgina.

Això ens permet veure a Homepage, sense canviar de pestanya, l'estat dels nostres serveis.

8.8 Personalització

Homepage és altament personalitzable. Algunes idees:

- **Títol personalitzat:** "BernatLab — el meu servidor".
- **Fons:** una foto de l'hort, una imatge de la Raspberry, un degradat subtil.
- **Icones personalitzades:** podem pujar les nostres icones (PNG, SVG) i referenciar-les per URL.
- **Bookmarks:** enllaços externs que volem tenir a mà (documentació de Docker, fòrums, etc.).
- **Widgets personalitzats:** podem escriure'ls nosaltres si tenim coneixements de JavaScript.

8.9 Integració amb altres serveis

Homepage es pot integrar amb molts serveis. Algunes integracions útils per al BernatLab:

- **Uptime Kuma:** ja l'hem vist.
- **Sonarr, Radarr, Plex, Jellyfin:** per a la gestió de continguts multimèdia.
- **Pi-hole:** per a la gestió de DNS i bloqueig d'anuncis.
- **Pihole, AdGuard Home:** per a DNS.
- **Nextcloud:** per a emmagatzematge al núvol.
- **Jellyseerr:** per a gestió de peticions multimèdia.

Tots aquests serveis tenen integracions preconfigurades o exemples a la documentació de Homepage.

8.10 Manuals de referència i resolució de problemes

Quan alguna cosa falla a Homepage, hi ha tres llocs on mirar:

1. **Logs del contenidor:** `docker logs homepage` o `docker compose logs homepage`. Aquí veurem errors d'inici, problemes de permisos amb el socket, errors de configuració.
2. **Documentació oficial:** gethomepage.dev. Molt completa, amb exemples de tots els widgets i serveis.
3. **GitHub Issues:** github.com/gethomepage/homepage. Sovint trobem algú que ha tingut el mateix problema.

Problemes habituals:

- **"Invalid host header":** hem de revisar `Homepage_ALLOWED_HOSTS`.
- **El socket de Docker no funciona:** cal revisar que estigui muntat correctament al `docker-compose.yml`.
- **Les icones no apareixen:** el nom de la icona no és correcte, o la URL no és accessible.
- **Els canvis no s'apliquen:** cal esperar uns segons, o forçar un refresh (`Ctrl+Shift+R`).

8.11 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
    subgraph Navegador["Navegador de l'usuari"]
        USR["Usuari obre http://100.115.134.76:3000"]
    end

    subgraph HomeContainer["Contenidor Homepage"]
        APP["App Next.js"]
        CFG["app/config/*.yaml"]
        SOCK["/var/run/docker.sock"]
    end

    subgraph Dades["Fonts de dades"]
        DOCKER["Docker Engine"]
        KUMA["Uptime Kuma API"]
    end
```



```
SIST["Sistema operatiu"]
end

USR --> APP
APP --> CFG
APP --> SOCK
SOCK --> DOCKER
APP -->|polling| KUMA
DOCKER --> SIST
```

8.12 Errors habituals

Error 1: "Invalid host header". Síntoma: la pàgina no carrega i veiem un error 400. Solució: afegir la IP/host a `Homepage_ALLOWED_HOSTS`.

Error 2: icones que no apareixen. Síntoma: les targetes es mostren sense icona, o amb una icona per defecte. Solució: revisar que el nom de la icona és correcte a la llista de icones suportades.

Error 3: widgets buits. Síntoma: la secció de widgets no mostra dades. Solució: comprovar els logs, revisar que el socket de Docker estigui muntat.

Error 4: canvis que no s'apliquen. Síntoma: editem un fitxer YAML, refresquem, i res canvia. Solució: validar la sintaxi YAML (un espai mal posat pot fer-ho fallar), esperar uns segons, refrescar forçadament.

8.13 Bones pràctiques

1. ****Configurar `Homepage_ALLOWED_HOSTS` correctament**** des del primer moment.
2. **Versionar la configuració** amb Git (la carpeta `data/homepage` conté només fitxers de configuració, ideals per a Git).
3. **Usar el socket de Docker en mode només lectura (`ro`).**
4. **Triar icones consistents.** Barrejar molts estils queda malament.
5. **Documentar els serveis** amb descripcions clares, per si algú altre ha d'entendre què fa cadascun.
6. **Fer còpies de seguretat** periòdiques de la carpeta de configuració.

8.14 Resum

Homepage és la porta d'entrada visual al BernatLab. Ens permet organitzar tots els nostres serveis en una sola pàgina, amb informació en temps real. La configuració és totalment basada en fitxers, cosa que ens permet versionar-la amb Git. El socket de Docker, en mode lectura, ens permet accedir a informació dels contenidors. `Homepage_ALLOWED_HOSTS` és una de les variables clau per evitar errors. En el proper capítol veurem com versionar tota aquesta feina amb Git i com mantenir una bona documentació.

8.15 Exercicis pràctics

1. Entra a `http://100.115.134.76:3000` i observa la configuració actual.

2. Mira el fitxer `services.yaml` dins de `/home/bernat/homelab/data/homepage/`. Quants grups té? Quants serveis?
3. Comprova la variable `Homepage_ALLOWED_HOSTS` amb `docker inspect homepage | grep -A 5 Env`. Què hi ha?
4. Afegeix un servei nou a `services.yaml` (per exemple, un enllaç a la documentació de Docker). Refresca la pàgina. Hauries de veure la nova targeta.
5. Mira els logs de Homepage: `docker logs homepage --tail 20`. Hi ha algun error?
6. Mira el widget de recursos del sistema. Quina és la temperatura actual? Quanta RAM s'està usant?
7. Si tens Docker socket, comprova que el widget Docker funciona i mostra els teus contenidors.

Comandes útils:

```
docker logs homepage
docker inspect homepage
ls /home/bernat/homelab/data/homepage/
ls /home/bernat/homelab/data/homepage/widgets/
nano /home/bernat/homelab/data/homepage/services.yaml
```

Paraules clau: **Homepage, panell, targetes, widgets, YAML, socket Docker, read-only, Homepage_ALLOWED_HOSTS, host validation, status page, integració, configuració.**

Capítol 9 — Git i documentació

*“Un servidor sense documentació és un misteri. Un servidor amb Git és una història.”**

9.1 Per què versionar la configuració

Al BernatLab tenim una quantitat creixent de configuració: fitxers `docker-compose.yml`, configuracions de serveis, scripts, documentació, decisions preses. Sense un sistema de control de versions, aquesta informació viu escampada: en fitxers, en missatges, en notes al mòbil, en la memòria. I quan arriba el moment de recordar per què vam prendre una decisió fa tres mesos, o de recuperar un fitxer que hem esborrat per error, no podem.

Git és el sistema de control de versions estàndard de la indústria. El fan servir des de projectes petits fins a Linux, passant per Microsoft, Google, Meta i qualsevol empresa tecnològica que puguis imaginar. És robust, distribuït (cada còpia és completa), gratuït, i molt ben documentat.

Al BernatLab, el farem servir per:

1. **Versionar la configuració** dels nostres serveis: tots els fitxers `docker-compose.yml`, configuracions YAML, scripts.
2. **Tenir un historial de canvis**: qui ha canviat què, quan, i per què.
3. **Recuperar versions anteriors**: si una actualització trenca alguna cosa, podem tornar enrere.
4. **Documentar**: els missatges de commit són una mena de diari del sistema.
5. **Treballar des de diverses màquines**: podem clonar el repositori des del PC, fer-hi canvis, i fer push quan estiguin validats.

9.2 Què versionar i què no

Abans de posar res a Git, cal tenir clar què hi va i què no:

SÍ versionar

- `docker-compose.yml` i tots els fitxers relacionats.
- Configuracions de serveis (YAML, JSON, TOML, etc.) que estiguin a `/home/bernat/homelab/data/*/config/...`
- Scripts de manteniment.
- `README.md`, `CHANGELOG.md`, documentació.
- `.gitignore`, `.gitattributes`.
- Plantilles de configuració (per exemple, `config.example.yaml`).

NO versionar

- **Secrets**: contrasenyes, tokens, claus privades. Això va a `.env`, que s'afegeix al `.gitignore`.

- **Dades binàries grans:** bases de dades SQLite, còpies de seguretat, fitxers multimèdia. Això va a `/home/bernat/homelab/data/` però NO a Git.
- **Caches i logs:** són regenerables.
- **Configuracions personalitzades** que canvien a cada màquina: fitxers amb IPs locals, noms d'usuari específics, etc.

La regla pràctica: **versionem la configuració, no les dades.**

9.3 Inicialitzar el repositori

Entrem a la carpeta de treball i inicialitzem un repositori Git:

```
cd /home/bernat/homelab
git init
```

Això crea una carpeta `.git/` amagada amb tota la infraestructura de versionat. A partir d'ara, Git començarà a seguir els canvis dels fitxers dins d'aquesta carpeta.

Comprovem l'estat:

```
git status
```

Ens mostrarà una llista de fitxers "untracked" — fitxers que encara no estan sota control de versions.

9.4 El fitxer .gitignore

Aquest és un dels fitxers més importants. Li diu a Git quins fitxers o patrons **ignorar** — no versionar, ni tan sols mostrar. La sintaxi és simple, un patró per línia:

```
# Secrets
.env
*.env
!.env.example

# Dades
data/
backup/
*.db
*.sqlite
*.sqlite3

# Logs
*.log
logs/

# Configuració local
.vscode/
.idea/
*.swp
.DS_Store

# Build artifacts
node_modules/
__pycache__/
*.pyc

# Caches
.cache/
```

Cal crear aquest fitxer **abans** de fer el primer commit. Si no, podem acabar versionant fitxers sensibles sense adonar-nos-en.

Al BernatLab, el `.gitignore` recomanat és:

```
# Secrets
.env
.env.*
!.env.example

# Dades
data/
backup/

# Logs
*.log

# Editor
.vscode/
*.swp

# OS
.DS_Store
Thumbs.db
```

Això ignora `.env`, totes les dades, els logs, i les configuracions locals d'editor. L'única cosa que es versiona són els fitxers de configuració i la documentació.

9.5 Primer commit

Ara podem afegir els fitxers al repositori:

```
git add .
git status
```

Això prepara tots els fitxers (excepte els ignorats) per al commit. `git status` ens hauria de mostrar una llista del que s'afegirà.

Si tot és correcte, fem el primer commit:

```
git commit -m "Configuració inicial del BernatLab"
```

El missatge de commit ha de ser clar i descriptiu. Aquest és el primer missatge, i marca l'inici de la història del projecte.

9.6 Estratègia de branques

Per a un homelab personal, la complexitat de les branques no cal que sigui gran. Hi ha dues estratègies raonables:

Opció A: una sola branca (main)

Tot va a `main`. Fem commits regulars, endavant. Si volem provar alguna cosa arriscada, podem crear una branca temporal, experimentar, i tornar a `main`. Però la majoria de canvis són petits i no justifiquen una branca.

Opció B: main + dev

Una branca principal (**main**) que sempre ha de funcionar, i una branca **dev** on anem acumulant canvis abans de validar-los. Quan estem segurs que tot funciona a **dev**, fusionem a **main**. Això ens dóna una mica més de seguretat.

Al BernatLab, **una sola branca és suficient**. La regla és: només fem commit quan els canvis funcionen. Si un canvi és arriscat, el fem servir per aprendre, i quan funciona, fem commit.

9.7 Flux de treball diari

El dia a dia amb Git al BernatLab segueix aquest patró:

```
# 1. Fer canvis (editar fitxers, crear-ne, esborrar-ne)
nano /home/bernat/homelab/data/homepage/services.yaml
```

```
# 2. Veure què ha canviat
git status
git diff
```

```
# 3. Afegir els canvis
git add .
```

```
# 4. Fer commit amb un missatge clar
git commit -m "Afegeix targeta Grafana a Homepage"
```

```
# 5. Tornar a la vida normal
```

Aquest patró el repetirem cada vegada que fem un canvi. Si treballem des del PC amb SSH, podem fer els canvis des de qualsevol lloc.

9.8 Missatges de commit clars

El missatge de commit és la **documentació del canvi**. Val la pena dedicar-hi un moment. Bones pràctiques:

- **Primera línia curta** (50-72 caràcters), en present, que resumeixi el canvi.
- **Línia en blanc**.
- **Cos del missatge** (opcional) amb més detalls.

Exemples bons:

Afegeix monitorització de Tailscale a Uptime Kuma

Afegeix un monitor de tipus ping a 100.115.134.76 amb interval de 5 minuts. Si Tailscale deixa de funcionar, rebrarem una alerta per Telegram.

Exemples dolents:

```
canvis
wip
probe coses
```

Un bon missatge de commit és or quan, al cap de tres mesos, volem entendre per què vam canviar aquella línia de configuració.

9.9 El fitxer README.md

A l'arrel de `/home/bernat/home\lab/`, el `README.md` és la **porta d'entrada** al projecte. Ha d'explicar:

- Què és el BernatLab.
- Com està organitzat.
- Com començar (clonar, instal·lar, configurar).
- Enllaços a la documentació (aquest manual, la carpeta `docs/`).

Un bon `README.md` té aquesta estructura:

```
# BernatLab

[Descripció breu]

## Estructura

[Llista de carpetes principals]

## Com començar

1. Requisits
2. Instal·lació
3. Configuració

## Documentació

[Enllaços a altres documents]

## Contacte

[Com trobar-me]
```

9.10 El fitxer CHANGELOG.md

Mentre que el `README.md` explica **què és** el projecte, el `CHANGELOG.md` explica **què ha canviat** al llarg del temps. És un registre cronològic.

```
# CHANGELOG — BernatLab

## 2026-07-08
- Afegit monitor de Tailscale a Uptime Kuma
- Canviat fons de Homepage
- Actualitzat Portainer a 2.21.0

## 2026-07-01
- Versió inicial amb Portainer, Uptime Kuma, Homepage
```

Aquest fitxer el mantenim a mà. No cal que sigui perfecte, només que reflecteixi els canvis importants.

9.11 Còpies de seguretat

Versionar amb Git ens dóna seguretat davant d'errors humans, però **no ens protegeix** de fallades de maquinari. Si la targeta microSD mor, perdem el `.git/` i, amb ell, tota la història.

Per això, al BernatLab tenim una política de **còpies de seguretat periòdiques**:

Què copiem

- Tota la carpeta `/home/bernat/homelab/` (configuració + dades + `.git`).
- Bases de dades d'Uptime Kuma, Grafana, etc.
- La carpeta `/home/bernat/homelab/backup/` allotja les còpies.

On copiem

Tres opcions:

1. **Un altre disc de la mateixa Raspberry** (per exemple, un SSD USB muntat a `/mnt/backup/`).
2. **Un altre ordinador de la xarxa** (per exemple, el PC, amb `rsync` o `scp`).
3. **Un servei al núvol** (Backblaze B2, Mega.nz, un repositori Git privat a GitHub).

La millor opció és una combinació: còpies locals ràpides + còpies al núvol periòdiques.

Com copiem

Un script senzill:

```
#!/bin/bash
# /home/bernat/homelab/scripts/backup.sh

DATA=$(date +%F)
DEST="/home/bernat/homelab/backup"

# Comprimir tota la carpeta homelab
tar czf $DEST/homelab-$DATA.tar.gz \
  -C /home/bernat \
  homelab \
  --exclude='homelab/backup/*.tar.gz' \
  --exclude='homelab/data/*/.git'

# Mantenir només les últimes 7 còpies
ls -t $DEST/homelab-*.tar.gz | tail -n +8 | xargs -r rm
```

Podem programar aquest script amb `cron` o `systemd` timers perquè s'executi diàriament, per exemple.

Restauració

Si hem de restaurar:

```
# Aturar serveis
cd /home/bernat/homelab
docker compose down

# Restaurar des de la còpia
tar xzf backup/homelab-2026-07-08.tar.gz -C /home/bernat

# Tornar a aixecar serveis
docker compose up -d
```

Aquesta és la operativa bàsica de recuperació de desastres. Sovint no cal restaurar-ho tot; n'hi ha prou amb un fitxer de configuració concret.

9.12 GitHub: el repositori remot

Per a una capa extra de seguretat i per poder treballar des del PC, podem **pujar el repositori a GitHub** (o a un altre servei similar: GitLab, Codeberg, Gitea autoallotjat).

```
# Crear un repositori nou a GitHub (sense README ni .gitignore, ja els tenim)
# Aleshores:
```

```
cd /home/bernat/homelab
git remote add origin git@github.com:bernatmora/bernatlab.git
git branch -M main
git push -u origin main
```

A partir d'ara, podem fer `git push` per pujar canvis i `git pull` per baixar-los des d'una altra màquina.

Per a informació sensible, podem fer servir **repositoris privats**. El pla gratuït de GitHub permet repositoris privats il·limitats.

9.13 Treballar des del PC

Si volem editar fitxers de configuració des del PC, podem clonar el repositori i treballar-hi:

```
# Al PC
git clone git@github.com:bernatmora/bernatlab.git
cd bernatlab

# Fer canvis amb el nostre editor preferit
nano services.yaml

# Pujar canvis
git add .
git commit -m "Canvia el fons de Homepage"
git push

# Aplicar els canvis a la Raspberry
ssh bernat@hortosona
cd /home/bernat/homelab
git pull
```

Aquest patró és molt poderós: ens permet treballar des d'un entorn còmode (el nostre PC, amb un editor potent), validar els canvis visualment, i només aplicar-los a la Raspberry quan estiguem segurs.

Alternativament, podem editar directament a la Raspberry per SSH, usant `nano` o un altre editor. Per a canvis petits, això és més ràpid. Per a canvis grans, treballar des del PC és més còmode.

9.14 Bones pràctiques

1. **Fer commits sovint**, amb missatges clars.
2. **Mai no versionar secrets**. Usar `.env` i `.gitignore`.
3. ****Documentar al README.md i al CHANGELOG.md****.
4. **Fer còpies de seguretat periòdiques** de tota la carpeta.
5. **Pujar a un remot** (GitHub, GitLab) per seguretat addicional.
6. **Revisar abans de fer commit**: `git status` i `git diff` són els nostres amics.

7. **Mantenir la història neta:** podem fer `git rebase` per reorganitzar commits locals abans de pujar-los.

9.15 Esquema del versionat

Esquema Mermaid (text):

```
graph TB
    subgraph Local["Carpeta local /home/bernat/homelab"]
        WORK["Working directory"]
        INDEX["Staging area"]
        REPO[".git/ (repositori local)"]
    end

    subgraph Remot["Repositori remot"]
        GH["GitHub (o similar)"]
    end

    subgraph Backup["Còpies de seguretat"]
        BACK["~/homelab/backup/*.tar.gz"]
        CLOUD["Núvol (Backblaze, Mega)"]
    end

    WORK -->|git add| INDEX
    INDEX -->|git commit| REPO
    REPO -->|git push| GH
    GH -->|git pull| REPO

    WORK -. ->|tar / rsync| BACK
    BACK -. ->|sync| CLOUD
```

9.16 Errors habituals

Error 1: versionar secrets per accident. Síntoma: pugem un `.env` amb contrasenyes a GitHub. Solució: configurar `.gitignore` des del primer moment; usar `git secret` o solucions equivalents per a informació crítica.

Error 2: fer commits amb missatges buits o inútils. Síntoma: la història és il·legible. Solució: dedicar 30 segons a escriure un bon missatge.

****Error 3: oblidar fer `git pull` abans d'editar**.** Síntoma: conflictes quan intentem fer push. Solució: sempre `git pull` abans d'editar.

Error 4: no fer còpies de seguretat. Síntoma: quan la microSD mor, perdem tot. Solució: backup periòdic, idealment al núvol.

Error 5: editar fitxers de dades directament. Síntoma: canvis que es perden en actualitzar un contenidor. Solució: editar fitxers de configuració, no pas dades binàries.

9.17 Resum

Git ens permet tenir un historial complet i versionat de la configuració del BernatLab, amb seguretat davant d'errors i la possibilitat de treballar des de múltiples màquines. Combinat amb bones pràctiques de `.gitignore`, fitxers `README.md` i `CHANGELOG.md`, i còpies de seguretat periòdiques, tenim un sistema robust i fàcil de mantenir. En el proper i últim capítol d'aquest mòdul, veurem què vindrà al BernatLab: la full de ruta dels pròxims mesos.

9.18 Exercicis pràctics

1. Entra a `/home/bernat/homelab` i comprova si ja hi ha un repositori Git (`ls -la .git`).
2. Si no n'hi ha, inicialitza'l: `git init`.
3. Crea un fitxer `.gitignore` adequat per a la carpeta.
4. Fes el primer commit: `git add . && git commit -m "Configuració inicial"`.
5. Crea un `README.md` i un `CHANGELOG.md` a l'arrel.
6. Prova de fer una modificació: edita un fitxer, fes `git status`, veu els canvis amb `git diff`, i fés commit.
7. Crea un script de backup a `scripts/backup.sh` i programa'l amb `cron`.
8. Si tens compte a GitHub, puja el repositori.

Comandes útils:

```
git init, git status, git add, git commit
git log, git diff
git push, git pull
git clone
git remote add origin URL
```

Paraules clau: **Git**, **control de versions**, **.gitignore**, **README.md**, **CHANGELOG.md**, **còpia de seguretat**, **rsync**, **tar**, **GitHub**, **repositori**, **commit**, **push**, **pull**, **homelab**.

Capítol 10 — Full de ruta del BernatLab

“Un servidor mai no està acabat. Està en camí.”

10.1 On som

A dia d'avui, el BernatLab té:

- Una Raspberry Pi 4 amb Debian 13 Lite, funcionant 24/7.
- Tailscale, que ens permet accedir-hi des de qualsevol lloc.
- Tres serveis principals: Portainer, Uptime Kuma, Homepage.
- La web pública Hort Osona allotjada a GitHub Pages.
- Una estructura de carpetes clara a `/home/bernat/home lab/`.
- (Imminent) un sistema de documentació i versionat amb Git.

Això ja és molt. Molts homelabs no passen d'aquí en anys. Però al BernatLab tenim plans: volem convertir-lo en el centre de tots els nostres projectes, especialment Hort Osona, els sensors LoRa, l'automatització, la música, la IA i el desenvolupament web.

En aquest capítol veurem què vindrà. No és un pla rígid — és una llista de prioritats que anirem abordant a mesura que tinguem temps, coneixement i ganes. Algunes coses es faran en setmanes; d'altres, en mesos o trimestres. I totes les prendrem amb calma.

10.2 Estratègia general

La nostra estratègia és **creixement incremental**: afegir un servei, entendre'l bé, documentar-lo, i només després passar al següent. Mai no afegirem dues coses noves alhora. Mai no despleguem un servei sense saber què fa i per què.

Això té tres avantatges:

1. **Redueix el risc d'errors**: cada canvi és petit i aïllat.
2. **Permet aprendre de veritat**: quan dediquem una setmana a entendre Node-RED, l'entendem.
3. **Genera confiança**: cada pas endavant és un pas que hem fet bé.

L'ordre de les prioritats ve determinat per:

- **Necessitat real**: el que ens serveix per als nostres projectes (Hort Osona, sensors).
- **Aprenentatge**: el que ensenya més i ens obre portes a més coses.
- **Complementarietat**: serveis que es connecten entre ells i creen un ecosistema útil.

10.3 Les pròximes passes, ordenades

A. Integrar sensors al Hort Osona (curt termini)

Aquesta és la prioritat número u. Tenim (o tindrem aviat) sensors al terreny — sensors de temperatura, humitat del sòl, il·luminació, potser més — que volem que enviïn dades al BernatLab. La integració es farà en diverses fases:

Fase 1 — Infraestructura MQTT

- Desplegar **Mosquitto**, un broker MQTT lleuger.
- Configurar-lo per acceptar connexions des de la xarxa Tailscale.
- Documentar el patró de temes (topics) que farem servir.

Fase 2 — Recepció i emmagatzematge

- Desplegar **InfluxDB** per emmagatzemar les dades dels sensors (time series).
- Desplegar **Telegraf** per recollir dades de MQTT i escriure-les a InfluxDB.
- Configurar la retenció de dades (quants mesos guardem, com agreguem).

Fase 3 — Visualització

- Desplegar **Grafana** per crear dashboards amb gràfiques de les dades.
- Integrar Grafana amb Homepage (afegir targeta al panell).
- Fer que les dades siguin visibles des del mòbil (Grafana és responsive).

Fase 4 — API pública

- Desplegar una **API REST** (probablement amb Node-RED o amb un petit servidor Python/Node) que llegeixi d'InfluxDB i serveixi dades agregades.
- Publicar l'API perquè la web Hort Osona la pugui consumir.

Fase 5 — Integració amb la web

- Modificar la web Hort Osona perquè mostri dades en temps real (o quasi-real) consumint l'API.
- Això ja és feina de desenvolupament web, no de servidor, però el servidor n'és la base.

B. File Browser — gestió de fitxers web

File Browser és una interfície web per explorar, pujar, descarregar i editar fitxers al servidor. És molt útil per:

- Gestionar les dades dels serveis sense entrar per SSH.
- Compartir fitxers amb altres dispositius de la xarxa Tailscale.
- Editar fitxers de configuració amb un editor web (per als moments en què no volem usar **nano**).

Es desplega fàcilment:

```
services:
  filebrowser:
    image: filebrowser/filebrowser:latest
    container_name: filebrowser
```

```
restart: unless-stopped
ports:
  - "8080:80"
volumes:
  - /home/bernat/homelab:/srv
  - /home/bernat/homelab/data/filebrowser:/database
```

Un cop configurat, ens donarà accés a tota la carpeta `/home/bernat/homelab` des del navegador. Compte amb els permisos: no volem que ningú que entri a File Browser pugui esborrar res crític per error.

C. Node-RED — automatització visual

Node-RED és una eina de programació visual, basada en fluxos, que ens permet connectar serveis, sensors, APIs, bases de dades sense escriure codi tradicional. Es representa com una graella on arrosseguem nodes i els connectem amb cables.

Al BernatLab, Node-RED ens servirà per:

- Processar les dades dels sensors (netejar, agregar, transformar) abans d'escriure-les a InfluxDB.
- Crear automatitzacions: "si la temperatura del sòl baixa de 5 °C, envia'm un missatge a Telegram".
- Connectar serveis: quan Uptime Kuma detecta una caiguda, executar una acció.
- Crear la base de l'API de Hort Osona: un node HTTP que respongui a peticions web.

Node-RED té una comunitat enorme i milers de nodes preconfigurats per a tota mena de serveis. El seu punt feble és que pot esdevenir un embolic de fluxos difícil de mantenir. Per això, caldrà documentar bé cada flux.

D. Mosquitto MQTT — el cor de la IoT

MQTT és un protocol de missatgeria lleuger, dissenyat específicament per a dispositius IoT. La seva filosofia és simple: un **broker** central (Mosquitto) rep missatges dels **publishers** (sensors) i els distribueix als **subscribers** (consumidors).

Ja n'hem parlat a la secció A. La diferència és que aquí el considerem com a servei independent, no pas com a part de la fase de sensors.

Característiques importants:

- Suporta QoS (qualitat de servei): 0 (com foc i oblida), 1 (almenys un cop), 2 (exactament un cop).
- Suporta retenció de missatges: l'últim valor queda disponible per als subscriptors nous.
- Suporta wildcards als temes: `sensors/+/temperature` selecciona tots els sensors.

Al BernatLab, el desplegarem amb una configuració que:

- Només accepti connexions autenticades.
- Restingeixi l'accés per ACL (l·listes de control d'accés).
- Exposeixi el port 1883 a la xarxa Tailscale.

E. InfluxDB — base de dades de sèries temporals

InfluxDB és una base de dades optimitzada per a **sèries temporals**: dades amb una marca de temps, com lectures de sensors, mètriques de sistema, preus, etc. A diferència d'una base de dades relacional, InfluxDB està dissenyada per:

- Emmagatzemar milions de punts de dades de forma compacta.
- Fer consultes ràpides sobre intervals temporals.
- Agregar dades (mitjanes, mínims, màxims) en temps real.

Al BernatLab, InfluxDB rebrà les dades dels sensors, les emmagatzemarà, i servirà per a Grafana. La configuració haurà de tenir en compte:

- **Retenció**: quant de temps guardem les dades en resolució original.
- **Continuous queries**: agregacions automàtiques (per exemple, "cada 10 minuts, calcula la mitjana horària i guarda-la").
- **Autenticació**: token d'accés per a Telegraf i Grafana.

Cal tenir present que InfluxDB pot ser exigent en recursos. Amb 4 GB de RAM, caldrà ajustar la configuració per no saturar el sistema.

F. Grafana — visualització de dades

Grafana és probablement la millor eina de visualització de dades de codi obert. Ens permet crear **dashboards** amb gràfiques, taules, gauges, heatmaps, alertes, i un llarg etcètera, connectant-nos a múltiples fonts de dades: InfluxDB, Prometheus, MySQL, PostgreSQL, fins i tot APIs HTTP.

Al BernatLab, Grafana ens permetrà:

- Veure l'evolució de la temperatura del sol al llarg del temps.
- Comparar la humitat de diferents zones de l'hort.
- Crear alertes visuals (un termòmetre que es posa vermell si la temperatura puja massa).
- Crear una **vista pública** que es pugui incrustar a la web Hort Osona.

Grafana té una corba d'aprenentatge inicial, però un cop entesa la lògica de panell-dashboard-data source, és molt potent.

G. PostgreSQL — base de dades relacional

PostgreSQL és la base de dades relacional de codi obert més avançada del món. Molts serveis la fan servir per defecte: Nextcloud, Mastodon, Gitea, etc. Al BernatLab, la podem necessitar per a:

- L'API de Hort Osona, si volem guardar informació estructurada (espècies plantades, calendari de sembra, etc.).
- Altres serveis que la requereixin per defecte.
- Aprenentatge: és una base de dades excel·lent per aprendre SQL.

Es desplega fàcilment amb la imatge oficial [postgres:16](#). Compte amb la configuració: cal establir una contrasenya forta per a l'usuari [postgres](#), i considerar si volem exposar el port 5432 a la xarxa (probablement no, només comunicació interna).

H. Integració LoRa SX1262 868 MHz

Aquesta és una de les parts més emocionants i complexes. **LoRa** (Long Range) és una tecnologia de comunicació per ràdio de llarg abast i baix consum. El mòdul **SX1262** treballa a la banda de 868 MHz (a Europa) i pot cobrir distàncies d'uns quants quilòmetres amb un consum molt baix.

Al BernatLab, l'objectiu és:

1. Connectar un mòdul SX1262 a la Raspberry (via SPI/GPIO).
2. Rep les transmissions dels sensors de l'hort (que tindran el seu propi mòdul SX1262).
3. Descodificar les dades i enviar-les per MQTT a InfluxDB.

Això implica:

- Hardware: mòdul SX1262, antena, cablejat.
- Software: una llibreria Python com [pyLoRa](#) o similar, o un servei com **ChirpStack** (un servidor LoRaWAN complet, però que pot ser excessiu per al nostre cas).
- Configuració: freqüència, amplada de banda, factor d'spreading, clau de xifratge.

Aquesta és la part més experimental del projecte. Hi dedicarem un capítol propi quan arribi el moment, perquè té molts detalls tècnics que no es poden tractar de passada.

I. Telegram — notificacions i comandament

Telegram és el canal de comunicació que ja estem fent servir per a Uptime Kuma. Però les seves possibilitats van molt més enllà:

- **Bot personalitzat:** podem crear un bot a mida que ens permeti interactuar amb el BernatLab: veure l'estat dels serveis, consultar dades de sensors, executar ordres.
- **Alertes avançades:** no només "servei caigut", sinó alertes riques amb imatges, gràfiques, botons d'acció.
- **Comandament remot:** podem enviar ordres al servidor des de Telegram (per exemple, "apaga el contenidor X" o "reinicia el servidor").

Per crear un bot, parlem amb @BotFather, igual que vam fer per a Uptime Kuma. Per fer-lo servir des de Python, podem fer servir la llibreria `python-telegram-bot` o `httpx` per cridar directament l'API de Telegram.

J. IA local — assistent Ollama

Ollama és una eina que ens permet executar **models de llenguatge grans (LLM) localment**, sense dependre de serveis al núvol. Al BernatLab, amb 4 GB de RAM, podem executar models petits com **Phi-3 mini**, **TinyLlama**, **Gemma 2B**, o **Llama 3.2 3B**. No seran tan potents com GPT-4, però funcionen, són privats, i són gratuïts.

Aplicacions:

- **Assistent personal:** podem integrar Ollama amb un client com **Open WebUI** (una interfície web similar a ChatGPT) per tenir el nostre propi ChatGPT privat.
- **RAG sobre documentació:** podem indexar les 76 fitxes d'Hort Osona (i tota la documentació del BernatLab) i fer que l'assistent pugui respondre preguntes sobre el contingut.

- **Resum automàtic:** podem fer que l'assistent ens resumeixi articles, correus, logs.
- **Generació de codi:** ajudar-nos a escriure scripts, configuracions, consultes.

Això s'integrarà amb Node-RED (per fer crides a l'API d'Ollama) i potser amb un client propi desenvolupat per nosaltres.

K. Desenvolupament web — VSCodium, Git, deploy

Per a la part de desenvolupament web, volem un entorn complet a la Raspberry:

- **VSCodium** (o Codi OSS) — un editor de codi accessible des del navegador, similar a VS Code. Es pot instal·lar amb **code-server**.
- Un **entorn de test** per a les webs que desenvolupem.
- Un **pipeline de deploy**: fer canvis, validar, publicar.

Això ja és un altre àmbit, però el BernatLab n'és la base.

10.4 Cronograma realista

No totes les fites es faran alhora. Una estimació realista, assumint que hi dediquem algunes hores cada setmana:

Fase	Què	Termini
1	Documentar el que tenim, Git, backup	Fet al Mòdul 1
2	File Browser	1-2 setmanes
3	Mosquitto MQTT, recepció bàsica	2-3 setmanes
4	InfluxDB, Telegraf, primers dashboards	3-4 setmanes
5	Node-RED, automatitzacions	4-5 setmanes
6	Grafana, visualització	5-6 setmanes
7	API pública, integració web	6-8 setmanes
8	LoRa SX1262 (experimental)	8-12 setmanes
9	PostgreSQL, base per a altres serveis	3-4 mesos
10	Telegram bot avançat	3-4 mesos
11	Ollama, Open WebUI, RAG	4-6 mesos
12	code-server, entorn desenvolupament	5-6 mesos

Això és aproximat. Algunes coses es faran abans, d'altres es retardaran. I apareixeran coses noves que no havíem previst. És la naturalesa d'un projecte viu.

10.5 Riscos i limitacions

Hem de ser honestos amb els límits del BernatLab:

Limitacions de maquinari

- **4 GB de RAM:** insuficients per a molts serveis simultanis. Si despleguem Grafana, InfluxDB, Node-RED, Ollama i mitja dotzena més, el sistema anirà just.
- **MicroSD:** com ja hem comentat, és un punt feble. A mitjà termini, caldrà migrar a SSD.
- **CPU ARM:** no tots els serveis tenen imatges arm64 natives. Cal verificar-ho.

Limitacions d'ample de banda

La pujada des de casa sol ser lenta (10-50 Mbps típicament). Si tenim molts dispositius accedint al BernatLab, podem saturar-la. La solució és Tailscale: el tràfic es comprimeix i xifra, però la velocitat de pujada segueix sent el coll d'ampolla.

Limitacions de temps

Un homelab és una afició, i com a tal competeix amb la feina, la família, la salut, el temps lliure. No podem posar-nos terminis inabastables. Cada pas endavant ha de ser sostenible.

Riscos de seguretat

A mesura que afegim serveis, ampliem la superfície d'atac. Cal:

- Mantenir tot actualitzat.
- Usar contrasenyes fortes.
- Configurar autenticació de doble factor on sigui possible.
- No exposar serveis a Internet directament.
- Fer còpies de seguretat regularment.

10.6 Què NO farem

Igual d'important que la llista de coses que farem és la llista de coses que no farem (almenys no immediatament):

- **Kubernetes:** excessiu per a un homelab petit. Docker Compose és suficient.
- **Alta disponibilitat:** no tenim dues Raspberry, no podem fer balanceig de càrrega. Acceptem que el servidor pot caure (i ens assegurem que sabem com recuperar-lo).
- **Streaming de vídeo o jocs:** la Raspberry no dona per a tant.
- **Allotjament massiu de webs:** tenim una sola màquina, no podem oferir serveis comercials.
- **Criptomineria:** ni se'ns acudeixi.

10.7 El mòdul 2 d'aquest manual

Quan hàgim completat les fases 2-6, estarem en disposició d'escriure el **Mòdul 2** d'aquest manual, que tractarà en profunditat:

- MQTT, broker, publicadors, subscriptors, wildcards.

- InfluxDB, llenguatge Flux, retenció, agregacions.
- Grafana, panells, dashboards, alertes.
- Node-RED, fluxos, nodes personalitzats, depuració.

Aquest Mòdul 2 s'escriurà quan tinguem experiència real amb tot plegat, no pas ara amb teoria.

10.8 El Mòdul 3: IoT i LoRa

Quan arribi el moment de desplegar LoRa, escriurem un Mòdul 3 dedicat a:

- Conceptes de ràdio: freqüència, amplada de banda, modulació.
- El xip SX1262: especificacions, comandes AT (si escau), biblioteques Python.
- Xarxes LoRaWAN vs. xarxes LoRa punt a punt.
- Integració amb el broker MQTT.
- Seguretat en LoRa: claus, xifratge, anti-replay.

10.9 El Mòdul 4: IA local

Quan Ollama estigui en marxa, escriurem el Mòdul 4 sobre:

- Què són els LLM i com funcionen a grans trets.
- Instal·lació i configuració d'Ollama a la Raspberry.
- Open WebUI com a interfície.
- RAG: dividir documents, indexar-los, recuperar-los.
- Casos pràctics: assistent per al BernatLab, resum de documentació, generació de codi.

10.10 Com prioritzem

Quan tot està per fer, és difícil saber per on començar. La regla és:

1. **Necessitem-ho per a un projecte real?** Si sí, va primer.
2. **Serveix de base per a altres coses?** Si sí, va primer.
3. **És divertit i fàcil?** Si sí, ens motiva per seguir.

La integració de sensors al Hort Osona compleix les tres: és per a un projecte real, serveix de base per a moltes coses (visualització, alertes, API), i és motivador.

10.11 Resum

El BernatLab és un projecte en construcció permanent. En aquest mòdul hem vist:

- On som avui.
- Cap a on volem anar.
- Quin ordre té sentit.

- Quins riscos i limitacions tenim.

Les pròximes passes són clares: File Browser, MQTT, InfluxDB, Node-RED, Grafana, API pública, LoRa, IA local, desenvolupament web. Cadascuna, quan arribi el moment, serà objecte d'un nou mòdul del manual.

10.12 Exercicis pràctics

1. Mira la pàgina `CHANGELOG.md` del BernatLab. Quin és l'últim canvi registrat?
2. Mira la carpeta `/home/bernat/homelab/`. Quins subdirectoris hi ha? Què hi falta segons la full de ruta?
3. Pensa en una millora que voldries afegir al BernatLab. Quin cost té? Quin benefici? Documenta-la al `CHANGELOG` amb l'etiqueta `[pendent]`.
4. Mira la llista de tasques pendents i tria'n una per fer aquesta setmana. Fes-la, documenta-la al `CHANGELOG`.
5. Comparteix el manual amb algú que conegui menys el tema i pregunta-li què no entén. Probablement trobaràs un capítol per millorar.

Comandes útils:

```
# Veure l'estructura
ls /home/bernat/homelab/
tree /home/bernat/homelab/ -L 2 # si tree està instal·lat

# Veure l'últim canvi al CHANGELOG
head -20 /home/bernat/homelab/CHANGELOG.md

# Comprovar espai
df -h
docker system df
```

Paraules clau: **full de ruta, MQTT, InfluxDB, Grafana, Node-RED, PostgreSQL, LoRa SX1262, Ollama, File Browser, Telegram, API, prioritització, cronograma, limitacions, sostenibilitat, mòdul 2, mòdul 3, mòdul 4.**

Final del Mòdul 1

Amb aquest capítol es tanca el primer mòdul del BernatLab. Hem après què és un homelab, com funciona la Raspberry Pi, com administrar Linux, com configurar xarxa i SSH, com desplegar serveis amb Docker, com gestionar-los amb Portainer, com monitorar-los amb Uptime Kuma, com presentar-los amb Homepage, com versionar-ho tot amb Git, i quin és el camí que tenim per davant.

El proper pas pràctic és posar en marxa el que s'ha descrit: instal·lar File Browser, configurar Mosquitto, muntar InfluxDB, desplegar Grafana, i començar a rebre les primeres dades dels sensors. I quan tot això funcioni, escriurem el Mòdul 2.

Fins aviat, BernatLab.